HandsFree: Development of a Deterministic Voice-Controlled Overlay for Android Navigation Using Accessibility Services

A Thesis Project
Presented to the Faculty of the
School of Computing
Holy Angel University

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Computer Science

Eriel Ben L. Baguio
Karl Shane Y. Bernarte
Juan Carlo C. De Castro
John Harold D. Doton

August, 2026

APPROVAL SHEET

This thesis entitled **"HandsFree: Development of a Deterministic Voice-Controlled Overlay for Android Navigation Using Accessibility Services,"** prepared and submitted in partial fulfillment of the requirements for the degree Bachelor of Science in Computer Science has been examined and is recommended for acceptance and oral examination.

Mary Jane C. Rabena,
Adviser

ORAL EXAMINATION
Approved by the committee of oral examiners on
[Date of defense]

Rinel Dela Cruz
Panel Chair

[Name of Panel Member]
Panel Member

[Name of Panel Member]
Panel Member

APPROVAL
Accepted and approved in partial fulfillment of the requirements for the degree Bachelor of Science in Computer Science.

Carisma A. Caro, MSIT
Program Chairperson, Computer Science

Marlon I. Tayag, DIT
Dean, School of Computing

Acknowledgement

The researchers wish to express their sincere gratitude to the people and institutions whose support made this study possible.

To our thesis adviser, Prof. Mary Jane C. Rabena, for the guidance, patience, and detailed feedback given throughout the development and writing of this study. The comments returned on each draft shaped both the direction of the research and the standard to which it was written.

To the developers and maintainers of the open-source projects on which this system depends, including the Whisper speech recognition architecture, the ONNX Runtime, the openWakeWord project, and the Filipino and Tagalog speech corpora used to fine-tune the recognition model, without which an offline system of this kind could not have been built within the timeframe of an undergraduate study.

Appreciation is also extended to the three technical evaluators who were invited to take part in the expert evaluation and who acknowledged the request, though circumstances prevented them from returning a completed assessment within the study period.

The researchers also express their genuine gratitude to the participating respondents. Their willingness to cooperate, share their time, and provide essential feedback during the data-gathering, system evaluation, and testing phases was absolutely vital to the validation and success of this study.

Finally, the researchers themselves would like to acknowledge one another, Eriel Ben L. Baguio, Karl Shane Bernarte, Juan Carlo De Castro, and John Harold Doton, for the

collective teamwork, cooperation, and contribution to the makings and completion of the study.

Laus Deo Semper!.

Table of Contents

Page
Cover Page..... i
Approval Sheet..... ii
Acknowledgement..... iii
Table of Contents..... iv
List of Tables..... v
List of Figures..... vi
Title Page..... 1
Abstract..... 2
Introduction..... 3-30
Review of Related Literature..... 6
Conceptual Framework..... 19
Objectives of the Study..... 25
Scope and Delimitations..... 27

Methods..... 30-60
Research Design..... 30
Sources of Data..... 34
Participants..... 35
Instruments..... 36
Data Collection..... 40
Data Analysis..... 42
Research Procedure..... 44
Ethical Considerations..... 58

Results..... 60-72

Discussion..... 72-80
Conclusion..... 76
Recommendations..... 78

Appendices

Appendix A RAD Model Lifecycle
Appendix B CRISP-DM Lifecycle
Appendix C Proposed Architectural Design of HandsFree
Appendix D End-User Survey Instrument (Google Forms)
Appendix E Expert Evaluator Survey Instrument (Google Forms)
Appendix F Informed Consent Form
Appendix G Standardized 52-Command Test Set

Appendix H Predefined Intent Vocabulary
Appendix I Experts' Curriculum Vitae
Appendix J Editor's Note
Appendix K Plagiarism Certificate
Appendix L Researchers Curriculum Vitae
Appendix M Algorithm Flowcharts
Appendix N Source Code and Build Instructions

List of Tables

Table 1 Word error rate before and after fine-tuning
Table 2 Overall command performance
Table 3 Command success rate by category
Table 4 Response latency by pipeline stage
Table 5 Wake-word detection scores
Table 6 Participant task performance by command
Table 7 SUS item response distribution

List of Figures

Figure 1 IPO Diagram
Figure 2 The Rapid Application Development (RAD) Model Lifecycle
Figure 3 The Cross-Industry Standard Process for Data Mining (CRISP-DM) Lifecycle
Figure 4 Implemented Architectural Design of HandsFree
Figure 5 Command success rate by category
Figure 6 Distribution of failures by pipeline stage
Figure 7 Root cause of each failure
Figure 8 How HandsFree matches a spoken command to an intent
Figure 9 How HandsFree finds the named control on screen

HandsFree: Development of a Deterministic Voice-Controlled Overlay for Android Navigation Using Accessibility Services

Eriel Ben L. Baguio
Karl Shane Y. Bernarte
Juan Carlo C. De Castro
John Harold D. Doton

Holy Angel University
Computer Science

August, 2026

Abstract

Mobile voice assistants remain unusable for many Filipino speakers because they depend on network connectivity and are trained predominantly on monolingual English speech, leaving Tagalog and code-switched Taglish commands poorly served. This study developed and evaluated HandsFree, an offline voice-controlled overlay for Android that operates through the Accessibility Service without holding any network permission, combining on-device speech recognition fine-tuned on Filipino speech, rule-based Tagalog and Taglish normalisation, and deterministic intent matching via Levenshtein edit distance. Fine-tuning reduced word error rate from 82.72% to 63.15% (a 23.7% relative reduction). Functional evaluation of 52 standardised commands produced a 67.3% command success rate and 32.7% Application Error Rate, with failures traced to unhyphenated Taglish verb forms and an inconsistency between edit-distance classification and literal keyword execution. An independent evaluation with fifteen users across 450 task attempts corroborated this: 75.1% succeeded outright, app launching succeeded in over 88% of attempts, and the system scored a mean System Usability Scale of 76.0, above the conventional average of 68. The system responded within approximately three seconds per utterance. These findings indicate that deterministic, fully offline voice control for Filipino, English, and Taglish is achievable on consumer Android hardware, with lexicon coverage as the principal remaining constraint.

Keywords: offline speech recognition, Filipino, Taglish, code-switching, Android Accessibility Service, deterministic intent matching, voice user interface.

Introduction

The growing prevalence of mobile devices as primary tools for communication and daily life management has brought the limitation of existing voice navigation architectures into sharper focus. For individuals who cannot easily use a touchscreen, such as those with visual impairments or limited hand mobility, voice control is not simply a convenience but a necessity. Users with visual or motor limitations encounter difficulty with specific interaction tasks, including locating on-screen elements, accurately tapping small controls, performing multi-step interactions, navigating unfamiliar interfaces, and interacting with applications that provide no dedicated voice-command interface.

Yet voice assistants available today fall short of fixing these problems. Mainstream assistants such as Google Assistant and Siri provide broad accessibility support, but their command processing is performed on remote servers and therefore depends on network connectivity. Their ability to operate arbitrary third-party application interfaces is also limited to applications that expose a compatible integration, rather than operating through the accessibility layer common to all installed applications (Pan et al., 2025; Torkamani & Zarin, 2025). Running the same processing directly on a phone avoids these problems in principle, but common smartphones lack the computer power to run the large AI models these assistants depend on (Wang et al., 2024). Taken together, these factors leave users who require consistent, always-available hands-free navigation, particularly those with visual impairments or motor limitations, without a dependable solution.

Existing hands-free navigation systems fall into two architectural categories, each carrying distinct operation limitations. Traditional voice control mechanisms require explicit API integration from individual application developers, which restricts their utility to pre-configured software and excludes the vast majority of third-party Android applications. More recent GUI agents, such as Mobile-Agent-v3 (Ye et al., 2025) and V-Droid (Dai et al., 2025), deploy Large Language Models and Vision-Language Models to automate screen interaction through probabilistic visual inference. While these systems achieve high benchmark accuracy under controlled conditions, Dong et al. (2025) identified a persistent "reasoning-execution gap" in which models correctly interpret user intent but consistently fail to translate that reasoning into accurate, deterministic physical touch coordinates. The result is hallucinated UI interactions, incorrect or phantom gestures that are fundamentally incompatible with the reliability demands of assistive technology. Systems such as Voicify (Vu et al., 2023) partially address universal navigation by leveraging Android accessibility metadata, but their reliance on deep learning parsers introduces inference delays that conflict with offline and privacy-first requirements.

Across both categories, existing solutions prioritize capability over execution determinism, leaving individuals with visual or motor impairments without a reliability applicable navigation tool. Application-specific integration excludes most third-party software outright. GUI agents introduce a documented reasoning-execution gap along with computational demands unsuited to lightweight, offline deployment. None of these approaches has been demonstrated for Filipino or Taglish speech, leaving deterministic,

offline, hands-free control unaddressed for users who communicate in these languages. This study addresses that combined gap directly.

The proposed HandsFree system addresses these limitations through a fully offline Speech-to-Intent architecture integrated with the native Android Accessibility Service API. Rather than generating probabilistic text responses or relying on visual inference, the systems extracts structured metadata directly from the active UI accessibility node tree, bypassing the interpretation errors common in generative models (Xie et al., 2021). A spoken command is converted to text on the phone itself, matched against a set list of recognized actions, and then the correct button is tapped automatically. The system covers everyday task areas including launching applications, screen navigation, messaging, alarm scheduling, web search, and answering incoming calls. Because every step happens locally on the device, no audio is sent to a server, and the system works even without the internet. The primary beneficiaries of this study are individuals with visual impairments, and privacy-preserving mobile navigation mechanisms that function reliably across all Android applications without requiring cooperation from third-party developers.

HandsFree makes use of its unique built-in language bridge. In everyday Filipino speech, it is natural to mix Tagalog and English in the same sentence, which is a style commonly called Taglish. Voice assistants like Google Assistant and Siri are not designed around this pattern, and their handling of code-switching mid-command is inconsistent, particularly for complex or rapid switching between languages (Mark Louis Lim et al., 2022). HandsFree allows this by running a command normalization step before anything

else: words and phrases in Tagalog or Taglish are automatically converted to their English equivalents, so that commands like "i-open ang Settings" and "buksan ang Settings" are treated the same as "open the Settings." This step is handled through a fixed word list rather than a large AI model, keeping the system fast and lightweight on standard phone hardware. Accordingly, this study develops and evaluates a fine-tuned Filipino and Taglish speech recognition model, a deterministic command-matching and execution pipeline built on Android Accessibility Services, and the resulting HandsFree Android application, and measures the system's word error rate, command success rate, usability, and technical quality through both researcher-administered testing and evaluation by end users and technical experts.

Review of Related Literature

Mobile Accessibility and Hands-Free Interaction

Hands-free interaction is not a convenience feature layered onto mobile computing; for a substantial population of users it is the only viable mode of access. Esquivel et al. (2024) found that people with disabilities depend on voice assistants for reminders, interface control, and environmental control specifically to preserve independent living, but that speech interpretation failures and cognitive load during use are severe enough to drive users to abandon the device altogether. Upadhyay et al. (2023) narrowed this further, showing that scheduling and alarm commands are among the most sustained everyday voice-assistant uses precisely because they eliminate the need to physically touch a device, yet the same users remain locked out of more advanced reminder functionality because those functions demand

precise, memorized phrasing. Ehtesham-Ul-Haque and Billah (2026) traced this precision requirement to a specific design pattern: across four commercial voice interface systems, fixed command formats, restrictive input timeouts, and insufficient feedback each independently degraded usability, and their proposed fix routes every command through a large language model to reconcile natural phrasing with rigid syntax.

Read together, these three studies describe a single unresolved contradiction rather than three separate findings: hands-free access is most necessary for the users least able to tolerate strict phrasing, yet the dominant fix in the literature (LLM-mediated reconciliation) reintroduces the cloud dependency, latency, and unpredictability that the Voice-Controlled Mobile Interaction and Offline Speech Recognition and On-Device Processing sections show to be the more fundamental architectural problem. This establishes the specific gap HandsFree occupies: a hands-free system that tolerates natural phrasing variation without requiring generative, cloud-routed interpretation to do so. HandsFree addresses the same rigidity Ehtesham-Ul-Haque and Billah identified through fuzzy matching against a predefined intent vocabulary, executed entirely on-device, rather than through an LLM intermediary, a distinction developed fully in the Offline Speech Recognition and On-Device Processing and Deterministic Rule-Based Voice Command Systems sections.

Voice-Controlled Mobile Interaction

Before evaluating how voice commands should be executed, it is necessary to establish voice as a general-purpose input mechanism for mobile task execution rather than a narrow query-response tool. Pan et al. (2025) characterize modern voice assistants as

task-executable , capable of invoking arbitrary system applications on a user's behalf rather than simply answering spoken queries, and note that this expanded scope is precisely what introduces privilege-escalation and inter-app privacy risk during execution. This reframes voice input as an execution layer sitting on top of a mobile operating system, not merely a transcription front-end: the voice assistant becomes an intermediary that must resolve a spoken command into an actual system-level action.

This reframing carries a direct implication for HandsFree's design: because voice is being used to execute actions across applications rather than to query a single assistant, the risk and reliability profile of how that execution happens (the Offline Speech Recognition and On-Device Processing, Android Accessibility Services, and AI-Based Mobile GUI Agents sections) matters as much as the accuracy of speech recognition itself. HandsFree adopts this task-executable model of voice input, but, as the following sections argue, resolves the execution step through fixed on-device rules rather than through the generative, cloud-dependent mechanisms that introduce the risks Pan et al. describe.

Offline Speech Recognition and On-Device Processing

The case for on-device processing rests on two independent bodies of evidence that converge on the same conclusion: cloud-centric voice execution is not a flawed implementation but a flawed foundation, and edge-based alternatives are already proven feasible on constrained hardware.

On the risk side, Shi et al. (2025) show that as GUI agents gain autonomy, their dependence on dynamic remote environments and action-observation loops makes them

highly susceptible to adversarial attacks and unintended contextual privacy leakage. Wang et al. (2026) attempted to mitigate this with GUIGuard, which sanitizes sensitive on-screen elements before screenshots are transmitted to the cloud, but concede that accurate privacy recognition remains a bottleneck, meaning the mitigation is only as reliable as the sanitization step it depends on. Chen et al. (2025) proposed a different mitigation, using contextual LLMs to analyze privacy policies and surface disclosure risk to users in real time, which manages the symptom of transmission risk rather than removing the transmission itself. Sun et al. (2026) formalized why these symptom-level fixes are insufficient: their OS-Sentinel framework demonstrates that cloud-based agents routinely execute unsafe operations and incur privacy violations specifically because no system-level constraint prevents them from doing so.

Across these four studies, sanitization, contextual alerting, and formal verification are all reactive layers wrapped around an architecture that still requires data transmission or continuous runtime monitoring to function safely. On the feasibility side, the literature shows this reactive layer is unnecessary because the alternative already works: Nelson et al. (2025) built a fully offline Speech-to-Intent voice assistant for commercial trucks with no internet dependency; Torkamani and Zarin (2025) demonstrated that an 8-bit quantized ASR model running on edge hardware improves real-time responsiveness while reducing thermal and computational load; Alexandridis et al. (2022) compressed a full ASR-plus-NLU pipeline into a roughly 2-megabyte footprint with TinyS2I; and Mekonnen et al. (2025) and Di Leo et al. (2025) each independently deployed offline voice recognition (via the VOSK engine) on

constrained hardware, a Raspberry Pi 5 and a real-time prototype, respectively, achieving zero external data transmission and sub-second latency.

Taken together, these two bodies of evidence do not merely support on-device processing as one option among several; they establish that the reactive-mitigation path (sanitize, alert, verify) is solving a problem that the feasibility studies show can be eliminated at the architecture level instead. HandsFree adopts the latter path. It integrates an offline speech-to-intent engine with native accessibility metadata and a preprocessing translation step that normalizes Tagalog and Taglish input to English before classification, so that every stage, audio capture, language normalization, intent classification, and gesture dispatch, resolves entirely on local hardware with no external network dependency at any point in the pipeline.

Android Accessibility Services

Android's Accessibility Service API is the mechanism that makes third-party, cross-application voice control possible without developer cooperation, and the literature establishes this capability from three complementary angles: extraction, monitoring, and gap-filling. Xie et al. (2021) established the base case, engineering an automated control tool that uses the Accessibility Service API to identify screen nodes and dispatch simulated gestures directly, without requiring any change to the target application. Arsan et al. (2025) extended this by showing the same API can continuously monitor interface events and extract structural view hierarchies during runtime, not just at a single capture point, meaning the metadata stays current as the screen changes. Zhang et al. (2021) addressed the resulting

gap in applications that expose incomplete accessibility metadata, developing a vision-based Screen Recognition model that infers accessibility properties directly from raw pixels when the API's own metadata is insufficient. Vu et al. (2023) then closed the loop for voice specifically: their Voicify system intercepts accessibility trees to map spoken input to on-screen targets, and their user study found that Android's own grid-based Voice Access was inefficient and imprecise because it required supplementary commands just to change grid granularity.

The throughline across all four studies is that the Accessibility Service API gives an external process, one with no special relationship to the target app's developer, a live, structured, and (with Zhang et al. 's complement) complete map of what is on screen and where. This is the exact technical foundation the HandsFree title depends on: it is what allows a voice command to be resolved into a physical action inside any installed app, not just apps built to support voice control. Where Vu et al.'s findings show that even Android's native grid-based approach is imprecise, HandsFree resolves commands directly against accessibility node identifiers rather than spatial grid coordinates, which is the deterministic targeting mechanism developed fully in the Deterministic Rule-Based Voice Command Systems section.

Voice-Based GUI Navigation and Existing Hands-Free Systems

Existing hands-free navigation systems split into two design philosophies, and comparing them clarifies what HandsFree does differently. The first philosophy, spatial-structuring, replaces sequential audio narration (which imposes heavy cognitive and

memory load on users with visual or motor impairments) with a fixed geometric framework. Gonzalez Peñuela et al. (2025) built TapNav, an adaptive spatio-tactile screen reader that uses physical markers to constrain input to specific screen regions, shifting cognitive demand from auditory memory to tactile recognition. Li et al. (2022) built TangibleGrid, which translates a visual layout into resizable tangible brackets on a physical grid baseboard. Lee et al. (2024) observed that this kind of tile-based spatial exploration aligns with natural spatial cognition, enabling non-linear navigation without extensive audio instructions. The second philosophy, direct semantic targeting, is represented by Vu et al.'s (2023) Voicify, which skips spatial structuring entirely and instead intercepts the accessibility tree so a user can name an on-screen element aloud and interact with it directly.

These two philosophies converge on the same conclusion from different directions: both sequential audio scanning and generic spatial grids introduce an intermediate step (cognitive scanning, or grid-granularity adjustment) between the user's intent and the target element, and removing that intermediate step is what improves usability. Voicify shows this can be done with accessibility metadata alone, without any physical hardware. HandsFree extends Voicify's approach rather than the tactile/spatial approach: the user names an on-screen element aloud, and the system resolves that name against the accessibility node tree directly, with no spatial grid and no physical marker required, with the added step that any Tagalog or Taglish wording in the command is normalized to English first (the Tagalog, Taglish, and Code-Switching in Voice Interaction section), so the same direct-targeting mechanism works regardless of which language the command was spoken in.

AI-Based Mobile GUI Agents

The dominant contemporary approach to autonomous mobile GUI control uses Large Language Models or Vision-Language Models to reason about a screen and generate an action, and the literature converges on a single diagnosis of this approach's core weakness: correct reasoning does not reliably produce correct execution. Dong et al. (2025) named this the "execution gap," showing that VLMs frequently reason correctly about user intent but consistently fail to translate that reasoning into precise, deterministic touch coordinates. Two subsequent systems attempted to close this gap without abandoning the generative paradigm. Ye et al. (2025) built Mobile-Agent-v3 as a multi-agent framework Manager, Worker, Reflector, and Notetaker agents coordinating to maintain memory and self-correct errors, which narrows the gap but introduces heavy computational overhead and multi-step inference delay incompatible with real-time use. Dai et al. (2025) built V-Droid, which repositions the LLM as a verifier rather than a generator, pre-evaluating a finite set of candidate actions extracted from the accessibility tree; this lowers latency close to real-time but the core execution remains probabilistic, since the LLM is still choosing among candidates rather than executing a fixed rule. Liu et al. (2025) attacked a different symptom, poor generalization to unseen scenarios, with LearnAct's few-shot demonstration framework, and Yang et al. (2025) quantified the ceiling on all of the above with the FingerTip 20K benchmark, showing that even the more advanced mobile LLM agents struggle to execute tasks deterministically without massive longitudinal contextual data.

The pattern across five independent research efforts, multi-agent collaboration, verifier-based filtering, few-shot demonstration learning, and benchmark diagnosis, is that each narrows the execution gap without closing it, because every one of these systems still resolves the final action through a probabilistic model rather than a fixed rule. This is the direct rationale for HandsFree's core architectural choice: rather than attempting to make a generative model's execution step more reliable, HandsFree removes the generative step from execution entirely. The Deterministic Rule-Based Voice Command Systems section defines exactly what replaces it.

Deterministic Rule-Based Voice Command Systems

None of the sources reviewed so far use "deterministic" as a formally defined term, but the property is what the Android Accessibility Services and AI-Based Mobile GUI Agents sections have been describing from opposite directions, and it needs to be named explicitly because it is the second load-bearing word in this thesis's title. In this research, a system is deterministic if, given the same recognized input and the same on-screen state, it always produces the same classification and the same output action, because every stage of the pipeline resolves through a fixed rule set and a closed, predefined vocabulary rather than through a model that generates or infers a response.

This definition draws a precise line against the systems reviewed in the AI-Based Mobile GUI Agents section. Dai et al. 's (2025) V-Droid still selects among LLM-scored candidate actions, the same fixed accessibility-tree input can yield a different scoring ranking depending on the model's internal state, so the system is probabilistic even though it is fast.

Ye et al. 's (2025) Mobile-Agent-v3 generates its next action from open-ended reasoning across multiple agents, which is generative by construction. Against these, the Android Accessibility Service literature in the Android Accessibility Services section already supplies the deterministic alternative without originally naming it as such: Xie et al. (2021) and Arsan et al. (2025) resolve actions against exact, enumerable node identifiers rather than inferred coordinates, which is a closed-set lookup, not an open-ended generation. HandsFree formalizes this as its execution model: a fixed intent vocabulary is matched against recognized speech using fuzzy string matching (Levenshtein distance) rather than a language model, and the matched intent is resolved to a specific accessibility node through direct tree lookup rather than probabilistic ranking. The same command spoken twice against the same screen state will therefore always dispatch the same gesture, a guarantee none of the generative or verifier-based systems in the AI-Based Mobile GUI Agents section can make, and the property that makes deterministic execution suitable for the low-latency, high-trust assistive use case established in the Mobile Accessibility and Hands-Free Interaction section.

Tagalog, Taglish, and Code-Switching in Voice Interaction

A voice system built for Filipino users must contend with code-switching as the default condition of speech, not an edge case to be handled separately. Montalan et al. (2025) demonstrated through BATAYAN, a Filipino NLP benchmark built with native-speaker input, that Filipino and Taglish remain under-represented in existing language models, and that even commercial AI systems perform poorly on Filipino-language tasks, meaning standard multilingual training is not sufficient for this context. Malaay and Ventura (2024) showed

what does work: a speech recognition system trained specifically on real Filipino-English broadcast speech reached 96.05% accuracy, and they attributed this directly to the inclusion of Taglish data during training rather than reliance on monolingual corpora. Lim et al. (2022) added a constraint that rules out a simpler fix: the frequency of code-switching within a single utterance does not affect recognition accuracy, so the problem cannot be resolved through calibration or threshold-tuning, a workable system must handle a language switch occurring at any point in a spoken command, not just at utterance boundaries.

These three findings combine into a single design constraint rather than three separate ones: Taglish is the norm, monolingual/generic-multilingual training is insufficient to handle it, and the switch point within an utterance is unpredictable, so any preprocessing step must normalize language before classification rather than attempting to classify mixed-language input directly. HandsFree implements exactly this constraint: before any voice command reaches intent classification, Tagalog and Taglish phrases are converted to their English equivalents through a fixed reference list, so that a command is interpreted consistently regardless of where in the utterance the language switch occurs, and, consistent with the Offline Speech Recognition and On-Device Processing section, this normalization step runs entirely on-device with no internet connection required.

Related Systems and Comparative Studies

The preceding sections established each of HandsFree's design choices individually against the specific literature that motivates it. This section places HandsFree side by side with the systems most directly comparable to it to make the cumulative difference explicit.

Commercial voice assistants remain the default point of comparison for any hands-free system, but both dominant platforms trade away the properties HandsFree is built around. Google Assistant and Siri both depend on cloud or hybrid cloud-NLU processing to interpret commands (Pan et al., 2025; Torkamani & Zarin, 2025), which reintroduces the transmission latency and privacy exposure that the Offline Speech Recognition and On-Device Processing section already established as the central argument against cloud-centric execution. Their third-party reach is also bounded by developer participation rather than by the operating system itself: Google Assistant's app-level control is largely limited to first-party services and apps that explicitly integrate with App Actions, and Siri's equivalent reach is limited to apps built with SiriKit, meaning neither assistant can control an arbitrary, unmodified third-party application the way an Accessibility Service-based system can. Neither is designed around code-switching as a default input condition, which the Tagalog, Taglish, and Code-Switching in Voice Interaction section established as the norm rather than the exception for Filipino users.

Against the research systems already reviewed in the Android Accessibility Services through AI-Based Mobile GUI Agents sections, the comparison is more direct because the citations are already established. Voicify (Vu et al., 2023) shares HandsFree's core mechanism, resolving voice input directly against the accessibility tree rather than through spatial grids or cloud NLU, and is fully on-device, but was not built to handle code-switched input, which is the gap HandsFree's Tagalog/Taglish normalization step closes. Mobile-Agent-v3 (Ye et al., 2025) and V-Droid (Dai et al., 2025) both achieve the same

unrestricted, developer-independent app control that Accessibility Service-based systems allow, but both remain LLM-dependent for execution: Mobile-Agent-v3 through generative multi-agent reasoning, and V-Droid through a probabilistic verifier scoring candidate actions. Both therefore inherit the reliability trade-off documented across the AI-Based Mobile GUI Agents section, neither can guarantee that the same command against the same screen state will always produce the same action, which is the deterministic property the Deterministic Rule-Based Voice Command Systems section defines and which HandsFree is built to preserve.

These five systems each satisfy some subset of on-device processing, deterministic execution, unrestricted third-party app control, and native Taglish handling, but none satisfies all four simultaneously, Voicify comes closest, lacking only code-switching support, while the LLM-based agents and the commercial assistants each give up determinism, offline capability, or both. HandsFree is positioned in the literature as the system that closes this specific combination of gaps, and the preceding sections each explain, from a different angle, why that combination is the one the evidence shows to matter for its target use case.

Conceptual Framework

INPUT	PROCESS	OUTPUT
• Raw Audio Input (Voice Commands in Tagalog, English, or Taglish) • Real-Time UI Metadata (captured via Android Accessibility Service API) • Predefined Command/Intent Vocabulary (fixed set of recognized commands, e.g. Open, Search, Call, Scroll, Set Alarm)	• Wake-Word Detection Stage • Noise Filtering & Audio Preprocessing • Offline Speech-to-Text (ASR Engine) • Rule-Based Tagalog/Taglish Command Normalization (Tagalog/Taglish → English) • Command Pattern/Fuzzy Matching (Levenshtein Distance) against Predefined Intent Vocabulary • DFS Node Locator • Coordinate Mapp)	• Deterministic Physical Interactions • Simulated Dispatch Gestures (Taps, Swipes, Scrolls) • App Launch Execution • Smart Search Query Dispatch • Alarm Scheduling (Set Alarm via Android System APIs) • Messaging Intent Recognition (recognized only, execution not yet implemented) • Measurable Research Outcomes (WER, Command Success Rate, Response Latency)

Figure 1 *IPO Diagram*

The HandsFree system operates on an Input-Process-Output (IPO) architecture in which every stage executes entirely on local device hardware. No audio data, screen metadata, or user information is transmitted to any external server at any point in the pipeline. The system's behavior is deterministic in the sense that every stage, from intent matching to UI action, operates on fixed rules and a closed vocabulary rather than on generative or probabilistic inference. Given the same recognized command and the same

on-screen state, HandsFree always produces the same classification and the same action, unlike systems that generate a response through open-ended language or visual reasoning. The revised framework below reflects the system's expanded scope, which now encompasses App Launching, Smart Search, Screen Navigation, Accepting Calls, and Scheduling, with Messaging supported at the intent-recognition level. It also incorporates the Rule-Based Tagalog/Taglish Command Normalization as a new processing component that normalizes multilingual Filipino input into English before intent classification.

Input Layer

The input layer consists of three inputs. The first is raw audio captured from the device microphone, which accepts spoken commands in Tagalog, English, or any combination of the two (Taglish). The system does not require the user to speak in a single language or to switch languages between commands; the Rule-Based Tagalog/Taglish Command Normalization, described below, normalizes all multilingual input transparently. The second input is real-time structural metadata continuously extracted from the active foreground application through the Android Accessibility Service API. This metadata exposes the underlying hierarchy of the active application, including element types, labels, and absolute screen coordinates, as structured machine-readable node data. The third input is a Predefined Command/Intent Vocabulary, a fixed, closed set of recognized commands and their associated intents, for example open, search, call, and scroll. This vocabulary is what makes the system behavior deterministic, rather than interpreting speech through open-ended

language understanding, the system only ever matches input against this fixed list, so its space of possible outputs is fully known in advance.

Process Layer

Wake-Word Detection Stage

When the Call Sign feature is enabled, the device microphone continuously monitors ambient audio for the wake phrase "Hey Jarvis" using a three-stage on-device detection cascade following the openWakeWord architecture: raw audio is converted into a mel spectrogram, encoded into an audio embedding, and evaluated by a keyword classifier trained to detect the wake phrase specifically. This detection stage is fully offline and deterministic, consistent with the rest of the pipeline. Upon detecting the wake phrase, the system activates and proceeds to the Audio Preprocessing Stage.

Audio Preprocessing Stage

Before the transcription of spoken commands can begin, the raw audio signal captured by the device microphone undergoes a preprocessing stage. This stage applies noise filtering techniques, specifically spectral subtraction and voice activity detection, to attenuate background acoustic interference and to isolate the speech segment from surrounding ambient noise. Preprocessing is a necessary architectural step for an offline system because, unlike cloud-based ASR engines that can leverage large server-side noise-robust acoustic models, the on-device ASR engine operates under memory and computational constraints that make it more sensitive to signal quality. By cleaning the audio input prior to

transcription, the preprocessing stage increases the accuracy of subsequent speech recognition and reduces the likelihood of misclassification downstream in the intent pipeline. This is particularly important for the target deployment context, where users may issue commands in variable acoustic environments such as public transit, workplaces, or homes.

The preprocessed audio is forwarded to the Offline Speech-to-Text engine, which transcribes the spoken command into text entirely on-device without transmitting audio to any external server. The speech-to-text component is a Whisper-tiny model fine-tuned on Filipino and Taglish speech, adapted specifically for this system rather than used in its pre-trained form, and executed entirely on-device.

The transcribed text is then passed to the Rule-Based Tagalog/Taglish Command Normalization, the architectural component that distinguishes HandsFree from existing assistants. Because spoken Filipino communication frequently mixes Tagalog and English within a single utterance, a phenomenon known as Taglish, the intent classification model cannot reliably process raw multilingual transcriptions without normalization. The Rule-Based Tagalog/Taglish Command Normalization addresses this by detecting Tagalog and Taglish tokens in the transcribed text and converting them to their English equivalents through a deterministic token-mapping vocabulary, producing a normalized English string as output. This normalized output is then passed to the NLP and Intent Classification stage for uniform processing.

The Intent Classification stage evaluates the normalized English transcription using Command Pattern/Fuzzy Matching (Levenshtein Distance), which compares the transcription against the Predefined Command/Intent Vocabulary described above and selects the closest matching intent within a defined similarity threshold. This accommodates natural pronunciation variations, regional accents, and minor disfluencies without requiring exact phrasing. Simultaneously, a Depth-First Search (DFS) algorithm traverses the captured accessibility node hierarchy to isolate actionable UI targets relevant to the recognized intent. A boolean guard condition prevents a new command from being captured while a previous utterance is still being processed, since each pipeline stage executes sequentially to completion before the next input is accepted. A Finite State Machine governs the pipeline lifecycle. The application moves through six states: Idle, Listening, Preprocessing, Normalizing, Processing and Executing, and every transition is validated against a fixed transition table. An activation is accepted only in the Idle state, so a second button press or a wake-word detection during processing is rejected rather than starting a parallel session. A watchdog returns the pipeline to Idle if transcription does not finish within ten seconds.

The Rule-Based Tagalog/Taglish Command Normalization: Multilingual Input Normalization

The Rule-Based Tagalog/Taglish Command Normalization is the component that enables HandsFree to accept voice commands in Tagalog, English, or Taglish without requiring the user to switch languages or to conform to a single linguistic register. Upon receiving the transcribed text from the offline ASR engine, the Rule-Based Tagalog/Taglish

Command Normalization tokenizes the string and evaluates each token against a bilingual lexicon that maps common Tagalog navigation-related terms and Taglish hybrid expressions to their English equivalents. Tokens that are already in English are passed through unmodified. Tokens identified as Tagalog or code-switched Taglish are replaced with their English counterparts. The resulting normalized English string is then forwarded to the intent classification stage.

This design decision normalizing multilingual input to a single target language before classification is architecturally significant for several reasons. First, it decouples language support from the intent model: adding a new supported language requires only updating the bilingual lexicon, not retraining the entire classification pipeline. Second, it preserves the deterministic properties of the Levenshtein Distance matching algorithm, which operates on a fixed English vocabulary and would otherwise require separate vocabulary sets per language. Third, it allows the system to handle mid-utterance code-switching, which is characteristic of natural Filipino speech, without requiring the ASR engine to be multilingual itself. The Rule-Based Tagalog/Taglish Command Normalization therefore functions as the architectural bridge between the multilingual realities of Filipino mobile users and the monolingual English processing pipeline that guarantees deterministic, offline intent classification.

Output Layer

System outputs manifest as deterministic physical interactions with the active application. Matched intents are mapped to absolute spatial coordinates from identified

accessibility nodes, and the accessibility service dispatches these as simulated gestures, triggering tap, swipe, or scroll events directly on the target application. The expanded output set now includes App Launch commands that open third-party applications by name, Smart Search queries dispatched to the device search interface, Messaging commands recognised at the intent level but not connected to message-sending execution in the current build, incoming Call Acceptance gestures, and Scheduling actions including creating alarms through Android system APIs. The entire process from voice input to gesture execution occurs without any cloud dependency and runs entirely on standard consumer-grade Android hardware. Beyond these functional actions, system output is also evaluated through measurable research outcomes: word error rate, command success rate, and end-to-end response latency, each detailed in the Methodology.

Objectives of the Study

This study aims to develop, implement, and evaluate HandsFree, an offline voice-controlled Android application that enables hands-free interaction across five core functions: application launching, searching, screen navigation, accepting incoming calls, and scheduling, with messaging recognised at the intent level but not connected to functional execution in the current build. The study will evaluate the system's voice-command processing, accessibility-based UI interaction, execution performance, offline operation, and usability among selected Android users and technical evaluators.

Specific Objectives:

- To develop and deploy an offline Speech-to-Intent (S2I) engine paired with a Levenshtein Distance algorithm to process voice input locally on the device. This engine utilizes fuzzy matching against a predefined intent vocabulary to handle natural speech variation, supported by a Rule-Based Tagalog/Taglish Command Normalization component that converts multilingual input to English before intent classification.

- To extract real-time structural metadata from active applications using the Android Accessibility Service API, leveraging a Depth-First Search (DFS) algorithm to traverse the view hierarchy and isolate actionable screen nodes for deterministic mapping of intents to absolute spatial coordinates.

- To design a modular system architecture supporting App Launching, Smart Search, Screen Navigation, Accepting calls, and Scheduling, with Messaging implemented at the intent-recognition level to allow for future functional expansion without requiring core pipeline redesign.

- To design a deterministic targeting mechanism that resolves spoken element names against the accessibility node tree, bypassing the generative interpretation errors common in probabilistic visual models and ensuring consistent interaction execution.

- To evaluate the completed system through structured usability testing with general Android users, assessing intent recognition accuracy, command normalization performance, processing latency, and overall usability as measured by the System

Usability Scale. Technical evaluation by Android development experts was planned as a complementary assessment but was not completed, as detailed in Participants and Results.

Scope and Delimitations

This study encompassed the development and evaluation of an offline voice-controlled Android application, limited strictly to the Android platform. The target hardware included standard consumer-grade smartphones prevalent in the Philippine market, specifically devices running Android 11.0 through Android 15.0. The system accepted voice commands in Tagalog, English, and Taglish, and a Rule-Based Tagalog/Taglish Command Normalization normalized all input to English before the intent classification stage so that the Speech-to-Intent engine could process commands consistently regardless of which language or combination of languages the user spoke.

The core functions in scope were App Launching, Smart Search, Screen Navigation, Accepting Calls, and Scheduling, where Scheduling covered creating alarms and countdown timers through Android system APIs. The system also supported hands-free activation through an always-on wake-word feature, disabled by default and enabled at the user's discretion. Messaging was included as a recognized intent within the command vocabulary but was not connected to functional message-sending execution in the current build. The feature set was designed to be modular, meaning individual functions could be added to or removed from the intent vocabulary without requiring changes to the core processing

pipeline or the node resolution layer. This modularity was a deliberate architectural decision rather than a limitation. The system leveraged the Android Accessibility Service API to extract active view hierarchies and a Depth-First Search algorithm to map interactive screen nodes to absolute spatial coordinates. Physical interaction was simulated through a predefined gesture vocabulary of single-finger taps, linear swipes, and basic scrolls, all executed without any external network dependency at any stage of the pipeline.

Several operational boundaries defined the limits of the proposed system. HandsFree explicitly excluded functionality within secure or protected graphical interfaces, as Android operating system protocols will intentionally block the accessibility service from reading metadata on screens that enforce system-level privacy flags; the application was therefore inactive during password input, banking operations, and DRM-protected media playback.

The Android Accessibility Service API, which the system depended on, was able to read view hierarchies and dispatch simulated gestures in standard third-party applications, but was not able to interact with system-level overlays that draw above all other layers, screens that explicitly disable accessibility metadata, or applications that use non-standard rendering pipelines that do not expose node data through the accessibility framework. Complex multi-touch interactions, such as pinch-to-zoom or multi-finger rotations, were excluded from the gesture vocabulary.

Scheduling functions were limited to creating alarms and countdown timers through Android system APIs; reading or checking existing entries, calendar event management, and

third-party scheduling applications fell outside the scope of this iteration. Messaging functionality was limited to intent recognition and classification only. The system did not implement functional message composition or sending in the current build, and this is documented as a known limitation rather than a completed feature. Voice capture quality depended on the microphone hardware of the host device, and the system was not able to compensate for the limitations of low-quality or damaged device microphones. Always-on wake-word activation was included as an optional feature that is disabled by default. The deployed keyword detector responds to a pre-trained third-party call sign rather than one trained on the application's own name. A custom "Hey HandsFree" wake-word model was attempted using the openWakeWord training pipeline. However, training a phrase-specific detector requires assembling a substantial positive dataset of the target phrase, typically recorded or synthesized across many speakers and acoustic conditions, alongside a matched negative dataset of non-wake-word speech to avoid false triggers. This data collection requirement could not be met within the project's remaining timeframe, and the attempt was not completed. The pre-trained third-party call sign was therefore retained for the deployed system.

System responsiveness and processing latency were bound by the localized hardware capabilities of the host device, meaning performance could vary on lower-tier smartphones with restricted processing power and memory. The offline speech recognition engine did not receive adaptive updates available to cloud-connected systems, and transcription accuracy

could be reduced in environments with significant background acoustic noise or when processing strong regional accents not well-represented in the engine's training data.

Speaker-disjoint splitting is standard practice in automatic speech recognition evaluation. Its absence in the primary source portion means the reported word error rate should be read as an upper bound on performance rather than as an estimate of generalisation to unseen speakers. The relative improvement over the baseline is unaffected, as both models were evaluated under identical conditions on identical data. Re-splitting on speaker identity and re-evaluating is identified as immediate future work.

Methodology

Research Design

This study employed a development research design with evaluation, appropriate for studies that produce a working system as their primary output and then assess that system against defined performance and usability criteria. This approach was suitable because the study's objectives required both building a functioning artifact, the HandsFree application, and empirically measuring how well that artifact performed once built, rather than testing a pre-existing system or purely observing an existing phenomenon (Shi et al., 2025).

A deterministic architecture, rather than a natural language processing or large language model based approach, was deliberately selected as the system's core design choice, for three reasons already established in the Review of Related Literature. First, deterministic execution guarantees that identical spoken input against an identical on-screen state always

produces the same action, a property NLP and LLM-based mobile agents cannot offer: Dong et al. (2025) documented a persistent "reasoning-execution gap" in which such models correctly interpret user intent but fail to reliably translate that interpretation into consistent physical actions. Second, the study's offline requirement excludes generative approaches as a practical matter, since state-of-the-art language and vision-language models capable of GUI reasoning require computational resources beyond what standard consumer-grade Android hardware can support on-device, whereas Levenshtein-based fuzzy matching against a closed vocabulary is lightweight enough to run in real time on such hardware. Third, a bounded and enumerable intent vocabulary allows system behavior to be fully specified and verified in advance, unlike the open-ended output space of a generative model, which cannot be exhaustively tested. These three properties, reproducibility, on-device feasibility, and verifiability, directly justify the deterministic design over an NLP-based alternative for this system's offline, accessibility-oriented use case.

The overall plan therefore proceeded in two connected phases: system development, in which the speech recognition model and Android application were built and iteratively refined, followed by the system evaluation, in which the completed system was tested for technical performance and assessed by end users and technical experts.

This study is classified as mixed-methods because it integrates two data types that answer different parts of the same research objectives. Quantitative data, word error rate, command success rate, and end-to-end response latency, answer the objectives concerned with how accurately and reliably the system performs. Qualitative and quantitative usability

data, gathered through the System Usability Scale and the Expert Evaluation Questionnaire, answer the objectives concerned with how usable and technically sound the system is from a human perspective. Neither data type alone would address all of the study's objectives: performance metrics cannot capture user experience, and usability scores cannot capture recognition accuracy. The two data types were integrated by first establishing system performance through quantitative testing, then interpreting that performance alongside usability and expert evaluation results in the Discussion, so that technical findings and human-centered findings are read together rather than reported as separate, disconnected results.

Software Development Methodology

The HandsFree Android application was developed following the Rapid Application Development (RAD) methodology (see Figure 2, Appendix A), which prioritized iterative prototyping over rigid sequential planning. This approach enabled continuous integration of interface adjustments in response to early constraints specific to the Android Accessibility Service API. RAD governed the construction of the application itself, its user interface, its accessibility service integration, and its command execution pipeline, but did not govern how the underlying speech recognition model was built, which followed a separate framework described below.

Machine Learning Development Framework

The offline Speech-to-Intent processing engine was developed following the Cross-Industry Standard Process for Data Mining (CRISP-DM) (see Figure 3, Appendix B), which governed the machine learning pipeline through a structured progression encompassing data understanding, intent vocabulary preparation, model training, and offline deployment evaluation. CRISP-DM was used specifically for the speech recognition model's development because, unlike the application's interface and control logic, the model's development required data collection, iterative fine-tuning, and accuracy evaluation cycles that RAD's prototyping approach does not account for.

RAD and CRISP-DM operated as two parallel but connected development tracks rather than two unrelated processes. The CRISP-DM track produced the fine-tuned speech recognition model as its output, and this model was then integrated into the RAD track as a component of the Android application. Interface-level adjustment made during RAD prototyping, such as changes to how recognized commands were passed to the application's command execution pipeline, in turn informed what output format the CRISP-DM track's model needed to produce. The two methodologies therefore intersected at the point of model integration, with CRISP-DM governing what happens before a command is recognized, and RAD governing what happens after.

Evaluation Method

Once development was complete, the system was evaluated through two complementary methods. Performance testing measured word error rate, command success

rate, and end-to-end response latency under controlled, researcher-administered conditions. Usability evaluation assessed the completed system through testing with general Android users using the System Usability Scale, and through technical review by Android development experts using a structured Expert Evaluation Questionnaire, Performance testing established whether the system functioned as designed, while usability evaluation established whether that functioning system was actually usable and technically sound from an external perspective, together addressing both the technical and human-centered objectives of the study.

Sources of Data

This study drew on two categories of data, distinguished by their origin. Secondary data consisted of two existing, publicly available Filipino speech corpora: SapinSapin and the Filipino (fil_ph) subset of Google FLEURS. These datasets were not generated by the research team; they were obtained from their respective public sources, combined, filtered, and split into training and test sets, and used to fine-tune the offline Whisper-tiny speech recognition model and to evaluate its word error rate against an unmodified baseline.

Primary data was generated directly by the research team through controlled functional testing and structured evaluation of the completed HandsFree application. This primary data is categorized into quantitative and qualitative types. Quantitative data consisted of objective performance metrics recorded during researcher-administered testing, including command success rate, Application Error Rate, and end-to-end response latency. Qualitative

data consisted of structured feedback gathered from end-user usability testing using the System Usability Scale from technical evaluation by Android development experts using the Expert Evaluation Questionnaire. Collecting both secondary and primary data allowed the study to assess model-level speech recognition performance and system-level usability as complementary parts of the same evaluation.

Participants

Fifteen participants completed the study. General Android users were recruited through purposive sampling within the researcher's academic institution and local community. All fifteen participants were treated as a single group and completed the identical ten-command task set and System Usability Scale, so no further subgroup division was applied. Eligible participants were required to be at least 18 years of age, utilize a smartphone as a primary tool for daily productivity or communication, and possess a device running Android 11.0 or higher. This eligibility criterion was restricted to Android users because HandsFree is built entirely on the Android Accessibility Service API, a platform-specific mechanism with no equivalent implementation on iOS or other mobile operating systems. The application cannot install, run, or expose any function on a non-Android device, since it is distributed as an APK file, a package format iOS does not support. Recruiting participants outside the Android ecosystem would therefore not have produced usable evaluation data, as they would have no way to install or run the application at all. The Android-only sample reflects the technical scope of the system under evaluation rather than a limitation of the sampling method.

None of the fifteen General Android Users in the primary sample reported a visual or motor impairment, despite this being the population the system's design was originally intended to serve. A supplementary accessibility follow-up was therefore conducted, in which three additional participants completed a qualitative follow-up questionnaire, two participants who reported a hand-mobility impairment affecting fine touchscreen precision, and one participant who reported a visual impairment (lazy eye) with an active effect on visual focus. A second individual initially considered for the visual-impairment category was excluded after follow-up clarification revealed no functional visual side effect from their condition, and despite continued recruitment effors, no further visual-impairment response is therefore treated as an illustrative case rather than a generalizable finding: findings from this follow-up are reported qualitatively in the Discussion section

Three technical evaluators were contacted and invited to complete the Expert Evaluation Questionnaire.

Instruments

Three instruments were used to collect the data required by the research objectives. The algorithms and system services used to build the application, including Levenshtein distance matching, depth-first search over the accessibility node tree, and the Android Accessibility Service itself, are architectural components rather than research instruments, and are described in the Research Procedure.

The first instrument is the application transcript log, developed by the researchers as an integrated component of the system. For every utterance it records four fields: the text produced by the speech recognition model, its normalised English form, the intent assigned by the classifier, and whether the resulting command executed successfully. It is applied to the system itself during controlled command testing rather than administered to a person. Data from this instrument are scored as the Application Error Rate, computed as the number of failed utterances divided by the total number of utterances attempted. Because each record is written automatically by the application without manual annotation or researcher interpretation, the same exported log yields an identical figure for any analyst, which establishes the reliability of the measure.

Two further measurement procedures are applied to the system rather than administered to a person. The first is offline word error rate evaluation, computed by comparing the fine-tuned speech recognition model's transcriptions against ground-truth reference transcriptions on a held-out test set drawn from the SapinSapin and Google FLEURS (fil_ph) corpora, both obtained via HuggingFace. This is not administered to a person; it is applied directly to the trained model. Word error rate is calculated using the standard formula, the sum of word substitutions, deletions, and insertions divided by the total number of words in the reference transcription, and is reported both as an aggregate score and broken down by source dataset. Because the calculation is deterministic given a fixed model checkpoint and test set, it produces an identical result on repeated computation, which establishes its reliability.

The second is end-to-end response latency, measured through automated timestamp logging within the application, recording the interval between command utterance and completed on-screen action for each tested command. This measure is applied to the system during the same controlled testing sessions used for the Application Error Rate, and results are reported as mean and range across all tested commands.

The second instrument is the System Usability Scale, a standardised ten-item instrument adopted without modification (Brooke, 1996). It measures perceived usability and is administered to general Android user participants. Participants self-administer the instrument through an online form immediately after completing the standardised task set, without a researcher present, in the display language they selected within the application. Responses are scored using the standard formula, in which odd-numbered items contribute the scale position minus one, even-numbered items contribute five minus the scale position, and the sum of converted scores is multiplied by 2.5 to yield a final score between 0 and 100. The instrument's validity and reliability are established in the published literature and were not re-assessed in this study.

Task scenario scripts provide standardised prompts so that all participants attempt the same commands under comparable conditions. The scenarios cover launching an application, conducting a web search, navigating the screen, and accepting or rejecting an incoming call. Messaging is excluded from the task scenarios, as it is not implemented in the delivered system and is stated as a delimitation. Scheduling, although implemented and exercised

during functional testing, was not included in the participant task set. A sample end-user survey instrument is provided in Appendix D.

The third instrument is the Expert Evaluation Questionnaire, a fifteen-item instrument developed by the researchers for this study and administered to technical evaluators through an online form. Items are distributed across three categories of five items each: Technical Accuracy, addressing the precision of voice command recognition and gesture execution; Mobile Usability and Design, addressing the interface against mobile design and accessibility considerations; and Practical Development Utility, addressing the system's suitability for its intended user base from a professional development perspective. Each item is rated on a five-point Likert scale from 1 as Strongly Disagree to 5 as Strongly Agree. Items in the Mobile Usability and Design category were informed by Google's published Android accessibility guidance and by the Web Content Accessibility Guidelines (WCAG) 2.2, together with the W3C's guidance on applying WCAG 2.2 to mobile applications. The instrument was not formally validated against an existing measure, and its results are therefore reported as structured expert judgement rather than as a validated psychometric instrument. A sample expert evaluation instrument is provided in Appendix E. While this instrument was developed and administered to three invited technical evaluators, none returned a completed response prior to the conclusion of the study, and its results are therefore not reported in this manuscript.

The transcript log and the expert instrument address different constructs and their results are reported separately rather than pooled. The transcript log records whether a

command executed correctly, which is an objective outcome generated by the system and independent of any observer. The corresponding expert item records a qualified evaluator's professional judgement of recognition quality, which is a subjective assessment informed by development experience. The two are treated as complementary: the first establishes what the system did, and the second establishes how a professional assessed the adequacy of that behaviour. Neither is used to validate the other.

System-based profiling recorded objective software performance during offline execution, capturing end-to-end processing latency and intent classification accuracy.

Data Collection

Data collection proceeded in two phases, corresponding to the two categories of data described in Source of Data: system-level performance data, collected without participant involvement, and participant-based evaluation data, collected from General Android Users and Technical Evaluators. The three invited technical evaluators did not submit completed responses despite confirmed receipt of the request, and no data from this instrument is included in the Results.

Phase 1: System-Level Performance Testing

This phase generated the quantitative, system-level data reported as word error rate, Application Error Rate, and end-to-end response latency, directly addressing the study's objectives concerning speech recognition accuracy and command execution reliability. The HandsFree application was installed on a target consumer-grade Android device. Researchers

executed a standardized set of voice commands across the core task categories, App Launching, Smart Search, Screen Navigation, Scheduling, and Accepting Calls, without participant involvement, to isolate system performance from variability introduced by different speakers. The application's internal transcript log automatically recorded each command's transcription, normalized text, assigned intent, and execution outcome, from which the Application Error Rate was computed. Response latency was captured through automated timestamp logging during the same testing sessions. Word error rate was computed separately, offline, by evaluating the fine-tuned speech recognition model against the held-out SapinSapin and Google FLEURS test sets, as described in instruments.

Phase 2: Participant-Based Evaluation

This phase generated the qualitative and participant-reported quantitative data addressing the study's objectives concerning usability and technical soundness, collected from the two participants groups described in Participants.

General Android Users first completed a brief orientation covering the application's predefined voice command vocabulary and its support for Tagalog, English, and Taglish input through the Rule-Based Tagalog/Taglish Command Normalization, using natural spoken commands. Immediately after completing the task set, participants self-administered the System Usability Scale through an online form. Responses were exported as CSV files for scoring using the standard SUS formula.

Data Analysis

Three measures in this study are each reported using the word "error rate" or "accuracy," and they are computed from different data and should not be conflated. Word Error Rate characterises the acoustic model in isolation, computed against the held-out test split of the SapinSapin and Google FLEURS speech corpora, a set of corpus speakers entirely unrelated to this study's participants. The Application Error Rate is a separate, system-level measure computed from the 52-command functional test described in Phase 1, conducted by a single member of the research team on one device, without any participant involvement. Participant task outcomes, reported in Table 6, are a third and distinct measure, drawn from the fifteen General Android Users who completed the task set during Phase 2. Because these three measures originate from different sources, corpus speakers, a single researcher-run test, and study participants respectively, a result reported under one measure does not describe the same population as a result reported under another.

Analysis followed the two-phase structure of the data collection procedure, with a stated computation for each measure. The transcript log was analysed by computing the Application Error Rate as the number of failed utterances divided by the total number attempted, giving a system-level measure of command success. Each failure was then classified by the pipeline stage at which it occurred, distinguishing utterances that reached the classifier and matched no intent from those that were classified correctly but could not execute, and further grouped by root cause. This classification was performed because the two failure types indicate different deficiencies and require different remedies.

Latency measurements were reported per pipeline stage and summed, in order to separate the fixed cost of model inference from the adjustable waiting period controlled by the voice activity detector. Wake-word scores were analysed by comparing the distribution of detection scores against the distribution of non-detection scores relative to the decision threshold, establishing the separation margin between them. Word error rate was compared against the unmodified baseline model on an identical held-out test set using identical decoding settings, so that the only variable between the two runs was whether the model had been fine-tuned.

System Usability Scale responses were scored using the standard formula, in which positively worded items contribute the scale position minus one and negatively worded items contribute five minus on the scale position, with scoring applied according to each item's actual polarity rather than its position in the questionnaire, since item order in the administered form does not follow the standard alternating sequence. The total of all ten converted scores is multiplied by 2.5 to produce a score from 0 to 100.

Expert Evaluation Questionnaire responses were analyzed using frequency distributions and mean scores across its three categories, Technical Accuracy, Mobile Usability and Design, and Practical Development Utility. Because the fifteen items were researcher-developed rather than adapted from a validated instrument, with the Mobile Usability and Design category informed by Google's published Android Accessibility guidance and the Web Content Accessibility Guidelines (WCAG) 2.2, results are reported

and interpreted as structured expert judgement rather than as scores from a formally validated psychometric measure.

Research Procedure

Rapid Application Development was selected because the system's requirements could not be fully specified in advance. The behaviour of the recognition pipeline against real Filipino and Taglish speech was unknown until it could be observed on a device, and each observation changed the requirements for the normalisation lexicon. A methodology built on short iterations producing a working prototype at each cycle was therefore appropriate, whereas a sequential methodology requiring complete specification before construction could not have accommodated requirements that emerged only through testing.

Each phase produced a defined output and progression to the next phase was governed by an explicit criterion. The requirements planning phase fixed the command scope at six intents and established the offline and deterministic constraints; progression required that the scope be agreed and the constraints stated. The user design phase produced the input-process-output framework, the system architecture, and the intent vocabulary; progression required that the architecture account for every command within scope. The construction phase produced the working overlay, and it was within this phase that CRISP-DM operated on the speech-to-intent engine; progression required that each in-scope command execute on a physical device. The cutover phase produced the tester build

incorporating consent, guidance and log export; progression required that the complete first-run sequence execute without failure on a clean installation.

User feedback shaped subsequent iterations through a defined loop. Each testing round produced a transcript log identifying utterances that failed to resolve. These were analysed to determine whether the failure arose in recognition, normalisation, classification or execution, and the corresponding correction was applied before the next iteration. Recognition failures traceable to vocabulary gaps were corrected by extending the normalisation lexicon rather than by modifying the matching logic, which preserved the deterministic guarantee across iterations.

CRISP-DM ran as a parallel workstream nested within the RAD construction phase. The acoustic model was developed as a data-mining workstream whose deliverable, a deployed ONNX model, became a component consumed by the wider RAD cycle. Recognition failures observed during RAD testing fed back as new data-understanding input to the CRISP-DM cycle, and each RAD iteration that depended on recognition accuracy consumed the most recent CRISP-DM deployment output.

The technical implementation of HandsFree followed a dual-model development framework reflecting its two parallel construction tracks: the Android application overlay, developed in Kotlin using Android Studio and Visual Studio Code, and the offline Speech-to-Intent processing engine. Each track was governed by a distinct methodology suited to its respective nature. The Android application overlay encompassing the DFS node

locator, the Rule-Based Tagalog/Taglish Command Normalization, the gesture dispatcher, and the floating overlay interface was constructed following the Rapid Application Development (RAD) methodology (see Figure 2, Appendix A). RAD prioritized iterative prototyping over rigid sequential planning, which was appropriate here because the dynamic constraints of the Android Accessibility Service API required continuous testing against live device environments. Initial development phases established the system requirements and defined the functional intent vocabulary. Developers constructed interactive prototypes that tested the deterministic coordinate mapping mechanism. The Rule-Based Tagalog/Taglish Command Normalization was developed and validated during these cycles, with the bilingual lexicon expanded iteratively based on observed misclassification patterns during testing. Continuous integration loops identified and resolved gesture dispatch anomalies throughout active development, with successive iterations refining both the overlay interface and the lexicon based on early functional feedback. The final phase stabilized the Kotlin codebase and prepared the application for formal usability evaluation.

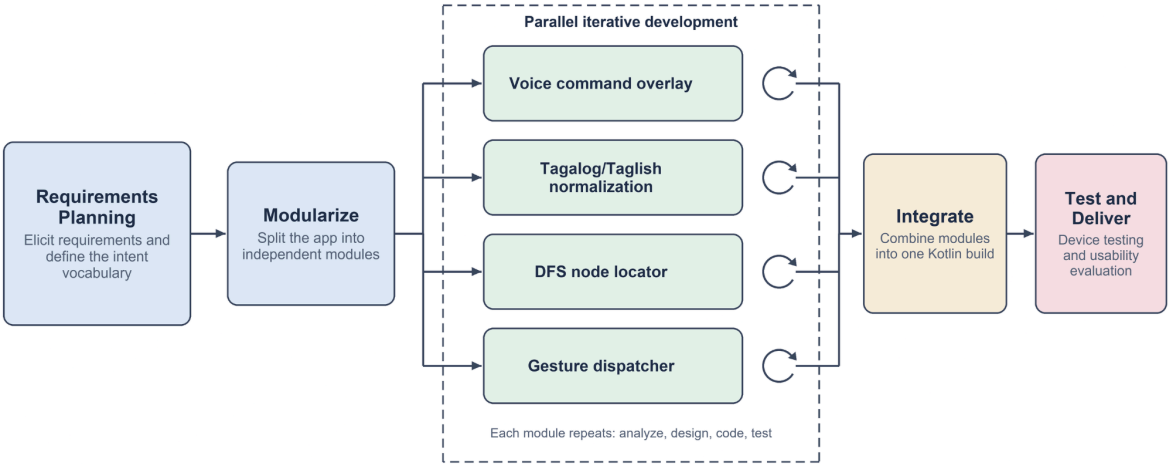


Figure 2 *The Rapid Application Development (RAD) Model Lifecycle as applied to HandsFree. Adapted from GeeksforGeeks (2026).*

The offline Speech-to-Intent (S2I) engine was developed in parallel following the Cross-Industry Standard Process for Data Mining (CRISP-DM) (see Figure 3, Appendix B). CRISP-DM governed the full model workstream shown in Figure 3: understanding the Filipino speech corpora, filtering the data, fine-tuning the Whisper-tiny Automatic Speech Recognition (ASR) acoustic model, evaluating word error rate, and exporting the model to ONNX for on-device deployment. The base model is openai/whisper-tiny (37.8M parameters), adapted through a full fine-tune, no LoRA or other parameter-efficient method was used. At this parameter count, a full fine-tune is tractable on consumer hardware, so parameter-efficient adaptation was unnecessary rather than rejected outright. The model was fine-tuned on a combined corpus drawn from the Filipino Speech Corpus and the Filipino subset of Google FLEURS, comprising 6,227 training and 1,479 held-out examples after filtering.

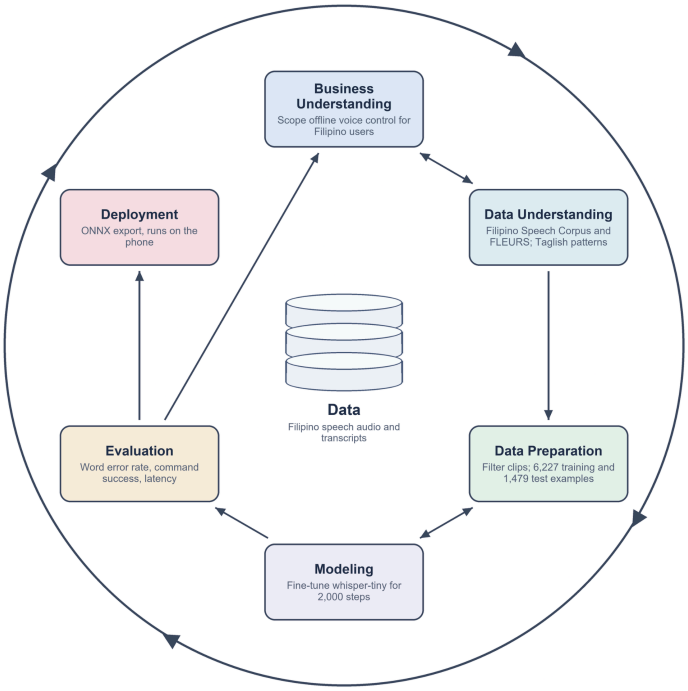


Figure 3 The Cross-Industry Standard Process for Data Mining (CRISP-DM) Lifecycle as applied to HandsFree. Adapted from Hotz (2025).

Within the CRISP-DM pipeline, the business understanding and data understanding stages defined the operational boundaries for offline voice command execution and catalog the speech patterns, linguistic registers, and Taglish code-switching behaviors characteristic of the target Filipino user population. The data preparation stage standardized raw audio inputs from both corpora and constructed the specific English intent vocabulary that served as the classification target after the Rule-Based Tagalog/Taglish Command Normalization normalized multilingual input. The modeling stage fine-tuned openai/whisper-tiny using HuggingFace's Seq2SeqTrainer for 2,000 steps (batch size 8, gradient accumulation 2, effective batch 16; learning rate 1e-5 with linear warmup over 50 steps; fp16 precision with

gradient checkpointing enabled; best-checkpoint restoration enabled). Training ran on a single NVIDIA RTX 3050 under PyTorch 2.11.0+cu128. The resulting transcriptions are then normalized through the Tagalog/Taglish-to-English lexicon and matched against the closed intent vocabulary using Levenshtein distance.

The evaluation phase measured fuzzy matching accuracy against held-out audio samples, assessed Rule-Based Tagalog/Taglish Command Normalization normalization coverage across Taglish test inputs and end-to-end processing latency on constrained consumer-grade Android hardware. The selected checkpoint was exported to ONNX (encoder.onnx, decoder.onnx, tokens.txt) and executed on-device through ONNX Runtime 1.19.0 using the CPU execution provider with two intra-op threads. NNAPI acceleration was deliberately not enabled, to keep on-device numerical behavior consistent with the evaluation environment. No large language model participates in the command path; intent resolution is deterministic, so identical input produces identical output.

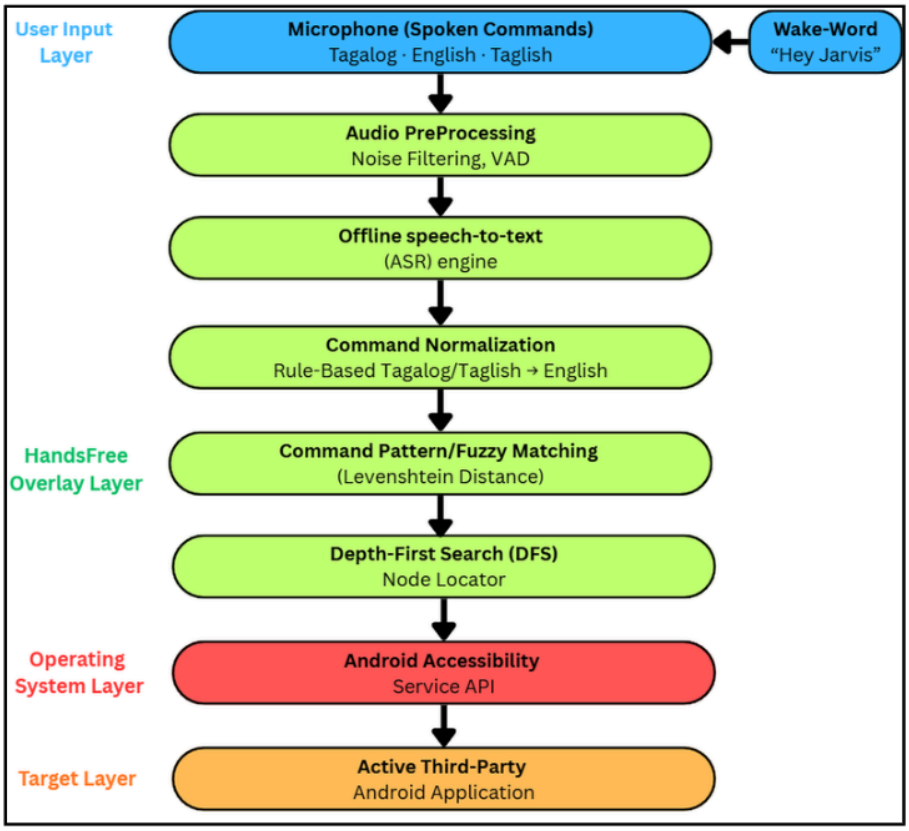


Figure 4 *Implemented Architectural Design of HandsFree*

Dataset Sources for Acoustic Model and Intent Evaluation

The training corpus combines two sources. The primary dataset is the Filipino Speech Corpus, retrieved from HuggingFace as sapinsapin/filipinospeechcorpus under an MIT licence. The upload comprises 305,246 segments from 124 speakers, spanning single-word utterances, read speech, spontaneous speech, and machine-generated speech. The upload does not document its collection methodology or original provenance; the corpus was therefore characterised directly from its own metadata fields. The dataset was not used as distributed. Three filters were applied to every row: a minimum of three words per utterance,

a minimum duration of 1.5 seconds, and exclusion of all segments marked as machine-generated. Only human speech was retained for training.

Figure 4 presents the revised proposed architectural design of the HandsFree system, organized into four distinct functional layers. Together these layers constitute a complete, closed-loop voice-to-gesture pipeline that operates entirely on local device hardware. The pipeline has been updated from the original design to include the Rule-Based Tagalog/Taglish Command Normalization as an explicit processing component within the HandsFree Overlay Layer, reflecting the system's expanded support for Tagalog, English, and Taglish voice input.

User Input Layer

The User Input Layer serves as the entry point of the system. This layer captures raw spoken commands through the device microphone, accepting voice input in Tagalog, English, or Taglish without requiring the user to select a language mode. When the optional Call Sign feature is enabled, this layer also performs continuous, always-on wake-word detection for the phrase "Hey Jarvis" using a three-stage on-device cascade following the openWakeWord architecture, before activating full command capture. Upon detecting a spoken command, or upon successful wake-word activation when Call Sign is enabled, the digitalized audio stream is passed downstream to the HandsFree Overlay Layer for preprocessing and processing. No audio data leaves the device at this stage or at any subsequent stage of the pipeline.

HandsFree Overlay Layer

The HandsFree Overlay Layer constitutes the core intelligence of the system and contains five sequential processing components. First, the audio signal passes through a Preprocessing Stage that applies noise filtering and voice activity detection to attenuate background acoustic interference and isolate the speech segment. Second, the cleaned audio is forwarded to the Offline Speech-to-Text (ASR) Engine, which transcribes spoken words into text entirely on-device using a fine-tuned Whisper-tiny model exported to ONNX, without transmitting audio to any external server.

Third, the transcribed text enters the Rule-Based Tagalog/Taglish Command Normalization, the new architectural component introduced in this revision. The Rule-Based Tagalog/Taglish Command Normalization detects Tagalog and Taglish tokens in the transcribed text and replaces them with their English equivalents through a deterministic bilingual lexicon, producing a normalized English string. This normalized output is what the subsequent classification components receive, ensuring that the intent matching pipeline operates on uniform English input regardless of the language in which the original command was spoken.

Fourth, the normalized English text is evaluated by the Command Pattern/Fuzzy Matching stage, which compares the text against the Predefined Command/Intent Vocabulary using Levenshtein Distance and selects the closest matching intent within a defined similarity threshold.

Fifth, the Depth-First Search (DFS) Node Locator receives the resolved intent, traverses the active accessibility node hierarchy, and identifies the specific UI element targeted by the voice command, mapping the resolved intent to an absolute set of screen coordinates.

Operating System Layer

The Operating System Layer interfaces directly with the Android Accessibility Service API, which serves as the bridge between the HandsFree overlay and the underlying Android operating system. This layer retrieves real-time structural metadata from the active application's view hierarchy including element labels, types, and bounding coordinates and provides this data to the DFS Node Locator. It also receives resolved spatial coordinates from the overlay layer and dispatches these as programmatic gesture events through the accessibility service framework, translating abstract intents into concrete physical interactions.

Target Layer

The Target Layer represents the active third-party Android application currently displayed on the device screen. This layer is the final recipient of the simulated dispatch gestures, taps, swipes, scrolls, or app-launch intents, generated by the system. Crucially, the target application requires no prior modification, custom integration, or developer cooperation to receive these simulated gestures. The Android Accessibility Service API allows HandsFree to interact with any foreground application universally, making the system

application-agnostic by design and extending its utility across the full ecosystem of Android applications without restriction.

System Algorithms

The user starts a command in one of five ways: the Start Listening button in the application, a floating microphone button that stays on screen above other applications, the system accessibility button, holding the volume-down key for about one second, or, when the Call Sign setting is enabled, speaking the wake phrase “Hey Jarvis”. The wake phrase is detected on the device by three small models in sequence, following the openWakeWord design. The first converts each 80-millisecond block of audio into a mel spectrogram, a representation of how much energy the sound has at each pitch. The second converts the spectrogram into a compact numerical summary of the sound, called an embedding. The third scores that summary for the wake phrase. A score of 0.5 or higher starts a command.

The microphone records at 16,000 samples per second and processes the audio in blocks of 1,024 samples, about 64 milliseconds each. Voice activity detection is based on loudness: a block counts as speech when its root-mean-square amplitude, a standard measure of average loudness, is above a fixed threshold. After speech has started, 16 quiet blocks in a row, about 1.02 seconds of silence, end the recording. A maximum listening window, 4 seconds by default and adjustable from 3 to 10 seconds in Settings, ends the recording if the user keeps talking or the room stays noisy. No noise-removal filter is applied; preprocessing consists of this speech detection only.

The recorded audio is transcribed on the device by the fine-tuned Whisper-tiny model, run through ONNX Runtime on the phone's processor. Whisper always pads its input to a 30-second window, so a short command and a long command take about the same time to transcribe. The output is plain text, for example "iklik ang settings".

The transcript is then normalised. It is lowercased and split into words, and each word is looked up in a bilingual lexicon that maps Tagalog and Taglish words to English. Before looking up a single word, the system first checks whether the word and the next word together form a two-word entry, such as "mga setting" for "settings". Words that are not in the lexicon are kept unchanged, since they are usually already English or are names such as YouTube. Filler words such as "ang" and "sa" map to nothing and are removed. For example, "iklik ang settings" becomes "click settings", and "buksan ang YouTube" becomes "open youtube". The lexicon is a fixed list, so the same transcript always produces the same English text.

The normalised command is then classified by fuzzy matching. Fuzzy matching means accepting a command that is close to a known phrase, not only one that matches it exactly; this is needed because the speech model often misspells words, and an exact match would reject many correct commands. HandsFree measures closeness with the Levenshtein distance (Levenshtein, 1966): the smallest number of single-letter changes, namely inserting a letter, deleting a letter, or replacing one letter with another, needed to turn one piece of text into the other. The distance between "scrol" and "scroll" is 1, because one letter is missing. The normalised command is compared with each of the 57 trigger phrases in the intent

vocabulary, which is grouped into six intents: application launch, navigation, web search, call handling, scheduling and messaging. For each phrase the system computes a score. If the command contains the phrase exactly, the score is 0. Otherwise the phrase is compared both with the start of the command and with the whole command, and the smaller distance is kept. The phrase with the lowest score wins, and it is accepted only if the score is 2 or less; otherwise the command is refused and the user is told it was not understood. For example, the transcript “scrol down” contains no trigger phrase, but its distance to “scroll down” is 1, so it is classified as navigation. No trigger phrase is shorter than four letters, because at a limit of 2 a very short phrase would match almost any input. Figure 8 in Appendix M shows this procedure as a flowchart.

After classification, the matched phrase is removed and the rest of the command is treated as its argument: “open youtube” gives the argument “youtube”. Each intent is then carried out as follows. Application launch compares the argument with the name of every installed application using the same edit distance and opens the closest one, allowing up to two changes, or one change for every three letters in longer names, so “yutube” still opens YouTube. Back, home, recent applications and notifications use the system’s own navigation actions, the same actions as the on-screen navigation buttons. Scrolling asks the first scrollable area on the screen to scroll, and falls back to a simulated swipe if no such area reports itself. Call handling first uses Android’s telephone service to answer or end a ringing call, and, if that permission is not granted, finds the Answer or Decline control in any visible window and presses it. Web search opens the Google results page for the spoken query in the

phone's browser. Alarms and timers are passed to the phone's Clock application with the spoken time or duration, for example "set alarm 8 PM" or "set 5 minute timer". Messaging is recognised but not carried out in the current build.

When the user names something to tap, for example "click settings", the system reads the accessibility node tree of the current screen. This tree lists every visible element, such as buttons, labels and text fields, with its text, description, identifier and position. A depth-first search (Tarjan, 1972) visits the tree one branch at a time, going as deep as possible down each branch before moving to the next, and visits the children of each element in order, first to last. Every visible element is scored against the spoken name. An element whose text or description equals the name scores highest; an element whose identifier equals the name, or whose text starts with or contains the name, scores lower. Elements that can be tapped, are enabled, or are buttons receive extra points, and elements below a minimum score are ignored. The search visits the whole tree and keeps the highest-scoring element, so the result does not depend on which match is seen first. If the whole name finds nothing, each word of four or more letters is tried on its own. If that also fails, a spelling-tolerant search compares each spoken word with each word on screen by edit distance, allowing one change for words of four or five letters and two for longer words, so "setings" still finds "Settings". The system then simulates a single tap at the centre of the chosen element. Figure 9 in Appendix M shows this search as a flowchart.

A finite state machine tracks the pipeline through six states: Idle, Listening, Preprocessing, Normalizing, Processing and Executing. Every change of state is checked against a fixed

table of allowed changes. A new activation is accepted only in the Idle state, so a second press or a wake-word detection during a command is rejected instead of starting a second command. A watchdog timer returns the pipeline to Idle if transcription does not finish within ten seconds. Because every step uses fixed lists, fixed thresholds and fixed rules, and no step generates or guesses a response, the same transcript on the same screen always produces the same action.

The complete source code of HandsFree, the trained model files and the build instructions are published in a public GitHub repository at https://github.com/JCDC0/HandsFree. The repository includes a Docker configuration that rebuilds the Android application package (APK) with the same build tools used in this study (see Appendix N).

Ethical Considerations

This section describes how participants, their privacy, and their data were protected throughout the research. Informed consent was obtained before any data were collected. Participants were presented with a consent screen within the application, and again within the survey instrument, describing the purpose of the study, the nature of the data collected, the voluntary character of participation, and the identity and affiliation of the research team. Participation proceeded only after explicit agreement. Participation was voluntary throughout. Participants retained the right to withdraw at any time without penalty and without providing a reason, and were informed that declining to participate carried no

consequence. Withdrawal could be effected simply by uninstalling the application, after which no further data of any kind could be produced.

This study posed minimal risk to participants. Task-based evaluation asked participants to issue voice commands to a smartphone application for a short, fixed set of scenarios; no risk beyond that of ordinary smartphone use was introduced. The main foreseeable discomforts were mild time burden from completing the task set and survey, and minor frustration in the event a voice command was misrecognized. These risks were minimized by keeping the task set brief, by allowing participants to skip or discontinue any task without penalty as described above, and by ensuring no sensitive on-screen content (passwords, banking interfaces, personal messages) was ever exposed to or processed by the researchers, since the application's accessibility metadata logging excluded protected screens by design.

Privacy and confidentiality were protected by the design of the application itself. Exported application logs contain command text only. No audio is recorded or retained at any point; speech is discarded immediately after transcription. The logs contain no names, contact details, message content, passwords, photographs, or location data. The application holds no network permission and therefore cannot transmit any data on its own; a participant transmits their log only by deliberately exporting and sending it.

Responses were collected through a process that ensured full anonymization. While initial raw data collection may have contained identifiers, all names and contact details were

removed and replaced with anonymized participant codes before the data analysis phase. This procedure ensures that the reported findings are strictly anonymized and that individual identities remain protected throughout the study. All research data, including participant survey responses and transcript logs, will be securely retained for a period of 3 years following the completion of the study, after which they will be permanently deleted.

Participant codes and exported response data were retained only for the period necessary to complete data analysis and the oral defense. Upon conclusion of these activities, all such data will be permanently deleted from the research team's devices and storage.

Results

Results for Objective 1: Development of the Offline Speech-to-Intent (S2I) Engine

The study produced a functioning offline voice-controlled Android overlay. The implemented system comprises an audio preprocessing stage with voice activity detection, an on-device speech recognition engine executing a fine-tuned Whisper model through ONNX Runtime, a rule-based Tagalog and Taglish normalisation layer containing 395 entries, a Levenshtein matcher operating against a closed vocabulary of six intents and 57 trigger phrases, and a command executor dispatching actions through the Android Accessibility Service.

Results for Objective 2: Implementation of Android Accessibility Service and DFS Node Resolution

Speech Recognition Model Evaluation

The fine-tuned model was evaluated against the untouched pre-trained `whisper-tiny` baseline on the identical 1,479-example held-out test set, using the same data collator, the same evaluation arguments, and the same word error rate computation. The only variable between the two runs was whether the model weights had been fine-tuned. The baseline model achieved a word error rate of 82.72%. The fine-tuned model achieved 63.15% at its best checkpoint (step 750 of 2,000), an absolute reduction of 19.57 percentage points and a relative reduction of 23.7%. Validation loss fell correspondingly from 4.06 to 0.73, corroborating the improvement independently of the word error metric. Word error rate improved consistently through step 750 and then fluctuated between approximately 63% and 73% for the remaining 1,250 steps without surpassing that checkpoint, a plateau consistent with mild overfitting on a corpus of this size. Because best-checkpoint restoration was enabled, the model retained for deployment is the step-750 checkpoint.

Model	Word Error Rate	Validation Loss
Baseline `openai/whisper-tiny`	82.72%	4.06
Fine-tuned, best checkpoint (step 750 of 2,000)	63.15%	0.73
Absolute reduction	19.57%	-
Relative reduction	23.7%	-

Table 1: *Word error rate before and after fine-tuning*

These figures should be interpreted with reference to the capability ceiling of the architecture employed. Liu et al. (2024) report Whisper-tiny averaging 109.53% word error rate across seven low-resource languages without fine-tuning; the present baseline of 82.72%

therefore falls below the published mean for this model class. PhoWhisper-tiny, the identical 39-million-parameter architecture fine-tuned on 844 hours of Vietnamese, attains 19.05%. The corpus used in this study is approximately two orders of magnitude smaller. The residual error is consequently attributable to corpus scale and model capacity rather than to the fine-tuning procedure, a reading reinforced by findings of Liu et al., that vanilla fine-tuning outperforms parameter-efficient alternatives at comparable data volumes.

The comparison with Malaay and Ventura (2024) requires similar qualifications. Their system performs continuous speech transcription using a Kaldi/WFST architecture with a domain-adapted language model, a fundamentally different task and pipeline. HandsFree does not require verbatim transcription; it requires the resolution of an utterance to one of six predefined intents, five of which execute device actions in the current build, a task in which Levenshtein-based fuzzy matching tolerates substantial word-level error.

Results for Objective 3: Modular Architecture and Functionality

The HandsFree architecture was successfully implemented as a modular suite of six distinct intents. This design allows for the independent update of functional handlers, such as App Launching or Scheduling, without affecting the core Speech-to-Intent pipeline. The implementation of a Rule-Based Command Normalization layer further supports this modularity by providing a consistent English interface for the intent matcher regardless of the input language register.

All six intents were implemented and five execute device actions; messaging resolves to its intent but does not execute, as stated in the delimitations. The application requires no

network permission and operates entirely on-device. Activation is available through the system accessibility button, a floating on-screen microphone button, holding the volume-down key for about one second, direct activation from the application interface, and an optional spoken call sign that is disabled by default.

Expert Evaluation Questionnaire: No data is reported for this instrument. Three invited technical evaluators acknowledged the request but did not submit completed responses within the study period.

Results for Objective 4: Performance of the Deterministic Targeting Mechanism

Word error rate characterises the acoustic model in isolation and does not describe the behaviour of the deployed system, in which three further stages, lexicon normalisation, fuzzy intent matching, and command execution, operate on the model's output. A functional evaluation was therefore conducted to measure system-level command performance directly. Fifty-two voice commands spanning nine command categories were executed on the target device using build 1.5.1-tester. Phrasing was varied within each category rather than a single phrase repeated, so that the evaluation measured the coverage of the normalisation lexicon rather than the recognition of one string. Each command was attempted once and failures were not retried. Every utterance produced a transcript record containing the recognised text, its normalised English form, the classified intent, and an explicit success or failure result, from which the metric was computed directly: Thirty-five commands completed successfully and seventeen failed, yielding a command success rate of 67.3% and an Application Error Rate of 32.7% (Table 2).

Measure	Value
Commands attempted	52
Completely successfully	35
Failed	17
Command success rate	67.3%
Application Error Rate	32.7%

Category	Attempts	Success	Fail	Rate
Search the web	6	6	0	100%
Open an application	6	5	1	83%
Alarm / Timer	6	5	1	83%
Answer / Reject a call	4	3	1	75%
Recents / Notifications	6	4	2	67%
Navigate back	6	3	3	50%
Navigate home	6	3	3	50%
Scroll	6	3	3	50%
Tap by name	6	3	3	50%
Total	**52**	**35**	**17**	**67.3%**

Table 3: *Command success rate by category*

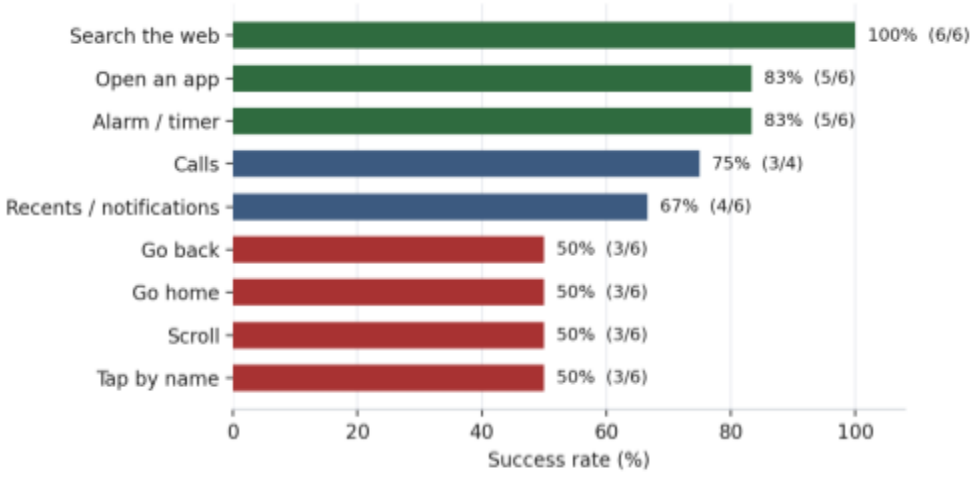


Figure 5: *Command success rate by category*

Performance varied by category (Table 3, Figure 5). Web search succeeded in all six attempts, while the four navigation categories that depend on literal keyword matching each succeeded in three of six. Of the seventeen failures, fourteen occurred because the transcribed utterance matched no entry in the intent vocabulary, and three because a correctly classified command could not be executed (Figure 6). The distinction is material: the former indicates a recognition or lexicon gap, the latter an execution gap, and the two require different remedies.

Three root causes account for all seventeen failures (Figure 7). Ten arose from transcription errors for which no lexicon entry existed. Four arose from a systematic pattern in which hyphenated Taglish verb prefixes were transcribed without the hyphen, so that "i-click" was recognised as "iklik" and matched no entry; within the tap-by-name category the split was

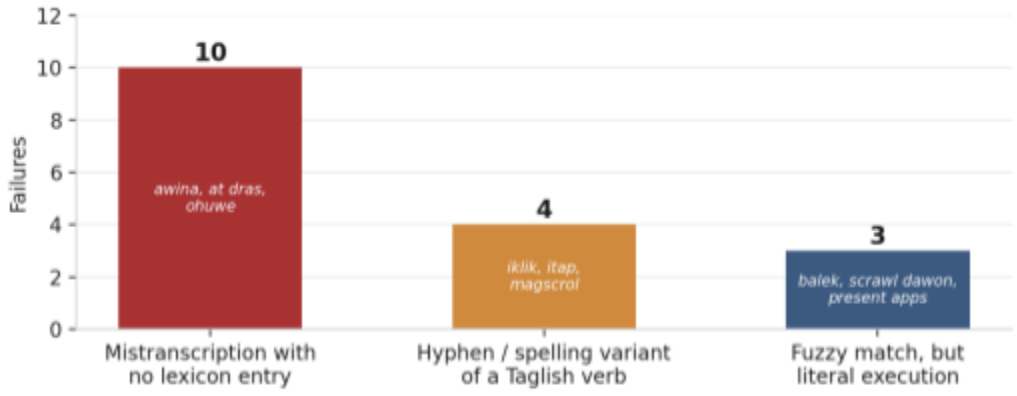


Figure 5: *Command success rate by category*

categorical, with all three native Tagalog forms succeeding and all three hyphenated Taglish forms failing. Three arose from a structural characteristic of the pipeline: the intent classifier tolerates edit distance, whereas the navigation handler tests for literal keywords, so an

utterance may be classified correctly and still fail to route to its handler. The evaluation was conducted by a member of the development team on a single device and therefore constitutes a functional evaluation of the implementation rather than a study of end-user performance.

These results indicate that the post-recognition stages of the pipeline recover a substantial proportion of acoustic model error. The clearest instance occurs in the web search category, which succeeded in all six attempts although no transcript was accurate: the model rendered "adobo recipe" as "adobore si pi" and "jollibee menu" as "jolly b. man yung", yet each command was classified and executed correctly because the trigger term survived transcription even where the argument did not. A word error rate of 63.15% therefore does not imply a comparable rate of command failure, and the observed 32.7% Application Error Rate is consistent with the architectural premise that a bounded intent vocabulary tolerates substantial word-level error.

Response Latency

Stage	Duration	Note
End of speech - voice activity detector fires	1.02s	Configurable - 16 chunks of 64ms
Voice activity detector - command result	2.08s	Dominated by Whisper inference
Total perceived response time	3.10s	From the user finishing speaking to the action running

Table 4: *Response Latency by pipeline stage*

End-to-end response time was measured on the target device across the two stages that dominate perceived delay. After the speaker stops talking, the voice activity detector waits a fixed trailing-silence window of approximately 1.02 seconds before declaring the utterance complete; inference and command execution then require a further 2.08 seconds, giving a total perceived response time of approximately 3.10 seconds (Table 4).

The inference cost is effectively constant regardless of utterance length, because the Whisper architecture pads every input to a fixed 30-second window before decoding. A one-second command and a ten-second command therefore incur the same inference time. This bounds what can be optimised: reducing the trailing-silence window shortens the wait, but the dominant term is fixed by the model architecture rather than by the application.

The trailing-silence window was reduced from 1.47 seconds to 1.02 seconds during development in response to tester feedback, and the maximum listening window was made user-adjustable between 3 and 10 seconds, with a default of 4 seconds, so that the value can be tuned to speaker behaviour rather than fixed by assumption.

Offline Operation

The application declares no `INTERNET` permission in its manifest and therefore cannot open a network connection under any circumstance, including at the request of its own code. This is verifiable by inspection of the installed package rather than by assertion. All speech recognition, normalisation, intent classification and command execution execute on-device. The single exception is web search, which hands a URL to the system browser;

the network request is then made by the browser, not by HandsFree. The application detects the absence of a validated connection and displays a warning before doing so, but still dispatches the request, since refusing to act on a connectivity check that may itself be wrong is the worst failure.

Wake-Word Detection Performance

Optional hands-free activation was evaluated over 150 seconds of continuous operation on the target device, sampling 62 detection windows. Four windows exceeded the decision threshold of 0.50, scoring between 0.8699 and 0.9943. The highest score recorded in

Measure	Value
Sampling window	150s continuous, 62 sampled windows
Detection scores	0.9931, 0.9943, 0.9433, 0.8699
Highest non-detection score	0.0232
Mean non-detection score	0.00055 (n = 58)
Decision threshold	0.50
Separation	Lowest detection + highest non-detection

Table 5: *Wake-word detection scores*

a non-detection window was 0.0232, with a mean of 0.00055 across the remaining 58 windows (Table 5). The weakest true detection therefore scored approximately 37 times higher than the strongest false signal, a separation wide enough that the threshold value is not sensitive to small changes.

Three detections handed off to the recognition pipeline correctly, and two proceeded to complete an application-launch command end to end. The cascade is deterministic: it evaluates a fixed sequence of convolutional models against a constant threshold, so identical audio produces an identical score, consistent with the determinism requirement applied to the rest of the system. One limitation remains. The deployed keyword classifier is a pre-trained third-party detector for the phrase “Hey Jarvis” rather than a model trained on the application's own call sign. Training a phrase-specific classifier requires a positive set of synthetic speech and, more importantly, a negative set drawn from the language the application is used in, in this case Tagalog, neither of which the project had time to assemble. Because only the final stage of the cascade is keyword-specific, substituting a custom call sign requires retraining a single small classifier rather than altering the architecture, and is identified as future work. The feature is disabled by default, as continuous operation holds the microphone open, which is a power and privacy cost the study does not impose on participants without an explicit opt-in.

Participant Usability and Task Performance

Fifteen general Android users completed the post-application survey. Each participant attempted all ten task-set commands three times, for a total of 450 command attempts, and rated the application using the ten-item System Usability Scale. Table 6 reports the aggregate task outcome for each command.

Command	Yes	No	Partly	Could not test
Buksan mo ang Youtube	43 (95.6%)	0 (0.0%)	2 (4.4%)	0 (0.0%)
Pakibuksan ang Facebook	40 (88.9%)	0 (0.0%)	5 (11.1%)	0 (0.0%)
Bumalik	37 (82.2%)	5 (11.1%)	3 (6.7%)	0 (0.0%)
Pumunta sa Home Screen	38 (84.4%)	3 (6.7%)	4 (8.9%)	0 (0.0%)
Mag-scroll pababa	38 (84.4%)	3 (6.7%)	4 (8.9%)	0 (0.0%)
Scroll pataas	38 (84.4%)	3 (6.7%)	4 (8.9%)	0 (0.0%)
Pindutin ang Calendar	31 (68.9%)	2 (4.4%)	12 (26.7%)	0 (0.0%)
Hanapin ang adobo recipe	27 (60.0%)	4 (8.9%)	11 (24.4%)	3 (6.7%)
Sagutin (while a call is ringing)	23 (51.1%)	7 (15.6%)	1 (2.2%)	14 (31.1%)
Tanggihan (while a call is ringing)	23 (51.1%)	5 (11.1%)	2 (4.4%)	15 (33.3%)
Overall	338 (75.1%)	32 (7.1%)	48 (10.7%)	32 (7.1%)

Table 6: Participant task performance by command (n = 45 attempts per command)

Across all 450 attempts, participants reported a command as fully successful 75.1% of the time and partially successful a further 10.7% of the time. The two call-handling commands, Sagutin and Tanggihan, recorded the highest proportion of "could not test" responses, 31.1% and 33.3% respectively, consistent with the practical difficulty of arranging a live incoming call during self-administered testing rather than with a system failure; excluding those responses, both commands succeeded outright in just over half of the attempts where testing was possible.

Table 7 reports the response distribution for each SUS item, scored according to the item's stated polarity: positive items contribute the scale position minus one, and negative items contribute five minus the scale position, consistent with the scoring procedure described in Instruments. The resulting mean SUS score across the fifteen respondents was 76.0, above the commonly cited average of 68 for the instrument.

Results for Objective 5: System Evaluation (Performance Metrics and Usability)

Statement	1	2	3	4	5
I think that I would like to use this system frequently.	1	1	1	10	2
I thought the system was easy to use.	1	0	2	5	7
I found the various functions in this system were well integrated.	1	0	3	4	7
I would imagine that most people would learn to use this system very quickly.	0	0	2	4	9
I felt very confident using the system.	1	1	2	5	6
I needed to learn a lot of things before I could get going with this system.	9	1	3	1	1
I think that I would need the support of a technical person to use this system.	6	5	2	2	0
I thought there was too much inconsistency in this system.	5	2	7	1	0
I found the system unnecessarily complex.	9	3	2	1	0
I found the system very cumbersome to use.	6	4	3	2	0

Table 7: SUS item response distribution (n = 15)

No response set rated every item identically across the ten statements, indicating that participants distinguished between the instrument's positively and negatively worded items

rather than defaulting to a single score, which supports the reliability of the aggregated ratings reported above.

Discussion

The findings indicate that the deterministic architecture achieves its intended purpose, and that the residual weakness of the system lies in acoustic recognition rather than in command interpretation. The clearest evidence appears in the web search category, which succeeded in every attempt although not one transcript was rendered accurately. The recognition model produced substantially corrupted text, yet each command was classified and executed correctly because the trigger term survived transcription even where the argument did not. A word error rate of 63.15% therefore does not imply a comparable rate of command failure, and the observed Application Error Rate of 32.7% is consistent with the architectural premise that a bounded intent vocabulary tolerates substantial word-level error.

Performance varied systematically by command category rather than randomly. Categories requiring only that a trigger term be recognised performed well, whereas categories requiring literal keyword matching within the navigation handler performed markedly worse. Analysis of the failures identified a structural cause: the intent classifier tolerates edit distance, but the navigation handler tests for literal keywords, so an utterance may be classified correctly and still fail to route to its handler. This represents an internal inconsistency in the treatment of recognition error and accounts for a distinct subset of the observed failures. After the evaluation, the same edit-distance tolerance was applied when

locating the on-screen target, so a slightly misrecognised control name no longer causes the tap to fail. This correction was not part of the measured build.

A second systematic pattern concerns hyphenated Taglish verb forms. The normalisation lexicon contained hyphenated entries, whereas the recognition model consistently produced the corresponding unhyphenated forms, which matched no entry. Within the affected command category the division was categorical: every native Tagalog verb form succeeded and every hyphenated Taglish form failed. This is a coverage gap in a hand-maintained resource rather than a defect in the matching algorithm, and it illustrates a broader property of the approach: a lexicon enumerated by hand does not converge, because Filipino generates inflected forms by rule while the lexicon must list them individually. Following the evaluation, the normalisation lexicon was extended with the unhyphenated and phonetic forms the recognition model actually produced (for example iklik, itap and ipindot), so these commands now resolve. The error rate reported above was measured before this correction and has not been re-measured.

Interpreting the SUS mean of 76.0 and the 75.1% task success rate alongside the technical metrics (WER of 63.15% and AER of 32.7%) provides a comprehensive view of the system's viability. These findings, when read together, fulfill the mixed-methods research design promise by explicitly connecting objective technical performance with subjective user perception. While the acoustic model retains a high error rate, the corresponding usability scores and success rates confirm that the deterministic intent-matching layer effectively bridges the gap between recognition accuracy and user experience.

The wake-word evaluation demonstrated that lightweight keyword detection can operate deterministically on-device with a wide separation between detections and background audio. The measured margin between the weakest true detection and the strongest non-detection indicates that the decision threshold is not sensitive to small variation, which is the property required for the feature to behave predictably.

These technical findings are consistent with how participants experienced the system directly. The mean System Usability Scale score of 76.0 across fifteen participants falls above the common average of 68, indicating that the recognition and matching error identified above did not translate into a poor overall user experience. This is consistent with the architectural premise discussed above: because the system tolerates substantial word-level error without requiring perfect transcription, participants were largely insulated from the acoustic model's underlying error rate.

Participant-based task performance across fifteen users and 450 attempts (75.1% succeeding outright, 10.7% partially) provides an independent check on the failure patterns identified above. The pattern for navigation commands held: app launching, which depends only on trigger-term survival, succeeded in over 88% of attempts, while commands routed through the literal-keyword navigation handler, back, home, and scroll, showed a consistently higher partial-or-failure rate of 15.6% to 17.8%, corroborating the structural inconsistency between the classifier's tolerance for edit distance and the navigation handler's requirement for exact matches. Search behaved differently under participant testing than under single-device researcher testing, however. Where the earlier functional evaluation found web

search resilient to transcription error because the trigger term survived regardless of argument accuracy, participant data shows search as the weakest-performing non-call command, with a 33.3% partial-or-failure rate. This divergence suggests that the trigger-survival explanation, while accurate for the specific transcription errors observed in single-device testing, does not generalize across the range of phrasing and acoustic variation introduced by fifteen different speakers, and indicates that researcher-run functional testing on a single device is an incomplete substitute for testing with real, varied users.

The absence of completed expert evaluations is a meaningful limitation of this study. Technical evaluation by experienced Android developers was intended to complement participant-based usability findings with an independent, professional assessment covering the system's architecture and implementation quality, as well as the underlying dataset and speech recognition model. The three invited expert evaluators acknowledged the request but did not respond with completed evaluations within the study's timeframe, so this review of the dataset and model was not obtained. Without this data, the study's evaluation of technical soundness relies solely on researcher-conducted functional testing and participant-reported usability, both of which are informative but do not substitute for expert review of the system's architecture, dataset, and model.

Accessibility Follow-up: Visual and Motor Impairment Perspectives

Because the primary sample did not include any users with a visual or motor impairment, a supplementary follow-up conducted with three participants recruited

specifically for this purpose. Both hand-mobility participants reported the voice control reduced the physical strain of repeated tapping, with one noting it was easier than fighting a hand tremor just to press a button, and both indicated that the app was helpful specifically because of their condition rather than something they would have found equally useful regardless. The visual-impairment participant, by contrast, reported needing more careful pronunciation and calibration for the app to respond correctly, and rated the experience as about equally useful whether or not their visual impairment was a factor.

This limited data suggest voice control's accessibility benefit may be more pronounced for motor impairments, where it substitutes an alternative for a physically difficult action, than for visual impairments, where the participant's main obstacle was recognition error rather than the input modality itself, the same transcription weakness already identified elsewhere in this study. Given the small, non-random nature of this supplementary sample, particularly the single visual-impairment response, these observations should be treated as directions for future study rather than established findings.

Conclusion

The study set out to develop and evaluate an offline voice-controlled Android overlay, and the following conclusions are drawn against each objective. First, the study produced a functioning offline voice-controlled Android application operating through the Accessibility Service. The implemented system integrates audio preprocessing, on-device speech recognition, rule-based Tagalog and Taglish normalisation, deterministic intent matching

against a closed vocabulary, and command execution, and requires no network permission of any kind.

Second, fine-tuning a compact speech recognition model on a combined Filipino corpus produced a substantial improvement over the unmodified baseline, reducing word error rate by 19.57 percentage points, a relative reduction of 23.7%. The residual error is attributable to corpus scale and model capacity rather than to the fine-tuning procedure.

Third, the deployed system resolved the majority of spoken commands correctly, achieving a command success rate of 67.3% and an Application Error Rate of 32.7% across 52 attempts. Performance exceeded what the underlying recognition accuracy alone would predict, confirming that the normalisation and matching layers recover a meaningful proportion of acoustic error.

Fourth, the system responded within approximately three seconds of the speaker finishing an utterance, with the dominant component being model inference, which is fixed by the architecture rather than by the application.

Lastly, offline operation was verified by inspection rather than assertion: the application declares no network permission and therefore cannot transmit data under any circumstance.

Overall, the study demonstrates that a deterministic, fully offline voice-control system for Filipino, English and Taglish is achievable on consumer Android hardware using a compact recognition model, and that a bounded intent vocabulary combined with rule-based

normalisation compensates for recognition error to a degree sufficient for practical hands-free operation. The system's principal remaining constraint is the coverage of its hand-maintained lexicon rather than the architecture itself.

Recommendations

The following recommendations arise directly from limitations and weaknesses identified in the results. Future work should secure committed technical evaluators before the evaluation period begins, confirming their commitment beyond initial acknowledgment of the request, and identify backup evaluators in advance to avoid the loss of expert evaluation data experienced in this study. This should include ensuring evaluators are able to review not only the application itself but also the dataset and speech recognition model, since that broader review was not obtained in this study. The Application Error Rate should be re-measured on the corrected lexicon, which now covers the unhyphenated Taglish verb forms that accounted for a distinct and preventable subset of the observed failures.

Edit-distance tolerance, which was added after the evaluation for locating named on-screen targets, should also be applied to the navigation keywords (back, home and scroll), which the handler still tests literally, so that a command classified correctly cannot fail because the handler requires a literal keyword match. Morphological normalisation should be implemented in place of hand-enumerated inflectional variants, since a substantial proportion of existing lexicon entries are inflected forms of targets already present, and rule-based handling would generalise to forms not yet encountered.

The training corpus should be re-split on speaker identity and the model re-evaluated, since the primary source portion exhibits speaker overlap between the training and test partitions, which means the reported word error rate should be read as an upper bound rather than as an estimate of generalisation to unseen speakers. A keyword classifier should be trained on the application's own call sign to replace the pre-trained third-party detector currently deployed. This study attempted such training using the openWakeWord pipeline but did not complete it within the available time, since generating a sufficient positive and negative training corpus for a wake phrase proved more data-intensive than initially anticipated. Future work should allocate dedicated time specifically for this data collection step, rather than treating it as a minor addition to the existing detection cascade.

An alternative architecture was also suggested: rather than transcribing speech to text and matching against the intent vocabulary after normalization, the system could instead perform matching directly on the acoustic signal itself, by passing the transcription and translation steps entirely. This approach was not implemented within this study's timeframe, since it requires assembling a new training corpus of audio paired directly to command labels and developing or adapting an audio-embedding matching model, a substantially different undertaking from extending the existing text-based pipeline. It remains a promising direction for future work, potentially reducing the compounding error introduced across the transcription, normalization, and matching stages of the current architecture.

Evaluation should be extended across multiple devices, speakers and acoustic environments, since the functional evaluation reported here was conducted by a single operator on a single handset under controlled conditions.

Finally, the deployed model's output should be verified against the training-environment reference implementation for identical input, a comparison that was not performed within this study and which would confirm that the on-device numerical behaviour matches the evaluated model.

References

Alexandridis, A., Sathyendra, K. M., Strimel, G. P., Kveton, P., Webb, J., & Mouchtaris, A. (2022). TinyS2I: A small-footprint utterance classification model with contextual support for on-device SLU. *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing*.

Arsan, D., Guo, C., Wellyanto, M. R., Ji, E., Talton, J. O., & Kumar, R. (2025). On-Device Interaction Mining. *Proc. ACM Hum.-Comput. Interact*.

Brooke, J. (1996). SUS: A “quick and dirty” usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189-194). Taylor & Francis.

Chen, C., Zhou, D., Ye, Y., Li, T. J., & Yao, Y. (2025). CLEAR: Towards Contextual LLM-Empowered Privacy Policy Analysis and Risk Generation for Large Language Model Applications. *Proceedings of the 30th International Conference on Intelligent User Interfaces*.

Cheng, J., Dankers, V., & Ponti, E. M. (2023). Towards low-resource code-switching speech recognition. arXiv preprint arXiv:2304.12397.

Conneau, A., et al. (2022). FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech. arXiv:2205.12446.

Dai, G., Jiang, S., Cao, T., Li, Y., Yang, Y., Tan, R., Li, M., & Qiu, L. (2025). Advancing mobile GUI agents: A verifier-driven approach to practical deployment. *arXiv preprint*.

Di Leo, S., De Cicco, L., & Mascolo, S. (2025). Real-Time Speech-to-Text on Edge: A Prototype System for Ultra-Low Latency Communication with AI-Powered NLP. *Information*.

Dong, L., Zhou, Z., Yang, S., Sheng, H., Cheng, P., Wu, Z., Wu, Z., Liu, G., & Zhang, Z. (2025). Say one thing, do another? Diagnosing reasoning-execution gaps in VLM-powered mobile-use agents. *arXiv preprint*.

Esquivel, P., Gill, K., Goldberg, M., Sundaram, S. A., Morris, L., & Ding, D.(2024). Voice Assistant Utilization among the Disability Community for Independent Living: A Rapid Review of Recent Evidence. *Human Behavior and Emerging Technologies*. https://doi.org/10.1155/2024/6494944

Ehtesham-Ul-Haque, M., & Billah, S. M. (2026). VoiceAlign: A Shimming Layer for Enhancing the Usability of Legacy Voice User Interface Systems. *Proceedings of the 31st International Conference on Intelligent User Interfaces (IUI '26)*, 1704–1721. https://doi.org/10.1145/3742413.3789167

Fairman, A. D., Indradhirmaya, F. A., Osal, R. B., & Saptono, A. (2025). Iterative User-Centered Design of the Mobile Device Assessment Tool (MoDAT). *Technologies*, 13(8), 358.

GeeksforGeeks. (2026, January 19). *Rapid application development model (RAD) - software engineering*. https://www.geeksforgeeks.org/software-engineering/software-engineering-rapid-application-development-model-rad/

Gonzalez Penuela, R. E., Liu, F., Macintyre, B., & Saffo, D. (2025). TapNav: Adaptive spatiotactile screen readers for tactually guided touchscreen interactions for blind and low vision people. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 9(4), Article 178.

Google. (2026). Accessibility. Android Developers. https://developer.android.com/design/ui/mobile/guides/foundations/accessibility

Hotz, N. (2025, January 2). CRISP-DM for data science teams: 5 actions to consider. Data Science PM. https://www.datascience-pm.com/crisp-dm-for-data-science-teams-5-actions-to-consider/

Le, T.-T., Nguyen, L. T., & Nguyen, D. Q. (2024). PhoWhisper: Automatic Speech Recognition for Vietnamese. *Proceedings of the ICLR 2024 Tiny Papers Track*. https://arxiv.org/abs/2406.02555

Lee, J., Peng, Y.-H., Herskovitz, J., & Guo, A. (2024). AltCanvas: A Tile-Based Editor for Visual Content Creation with Generative AI for Blind or Visually Impaired People. *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility*.

Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady, 10*(8), 707-710.

Li, J., Yan, Z., Jarjue, E., Shetty, A., & Peng, H. (2022). TangibleGrid: Tangible Web Layout Design for Blind Users. *The 35th Annual ACM Symposium on User Interface Software and Technology (UIST '22)*.

Lim, M., Xu, A., Lin, C., Chen, Z., & Pascual, R. (2022). Developing an automatic speech recognizer for Filipino with English code-switching in news broadcast. Proceedings of the 2022 14th International Conference on Knowledge and Smart Technology, 13-17. https://doi.org/10.1109/KST53302.2022.9727235

Liu, Y., et al. (2025). LearnAct: Few-Shot Mobile GUI Agent with a Unified Demonstration Benchmark. *arXiv preprint*.

Mager, M., Gutierrez-Vasques, X., Sierra, G., & Meza-Ruiz, I. (2018). Challenges of language technologies for the indigenous languages of the Americas. Proceedings of the 27th International Conference on Computational Linguistics, 55–69.

Malaay, E. M., & Ventura, R. A. (2024). Filipino-English Continuous Speech ASR: Towards Application of a Closed Captioning System for Philippine TV News Broadcast. *J-STAR*, 24.

Mekonnen, F. Y., Al Bataineh, M. F., Abu Abdoun, D., Serag, A., Tamiru, K. T., Abula, W., & Darota, S. (2025). Development of a fully autonomous offline assistive system for visually impaired individuals: A privacy-first approach. *Sensors*.

Montalan, J. R., et al. (2025). Batayan: A Filipino NLP benchmark for evaluating Large Language Models. Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. https://aclanthology.org/2025.acl-long.1509/

Nelson, R. D. O., De Almeida, L. G., & Levenhagen, I. A. (2025). An AI-based embedded offline voice assistant for commercial trucks: Enhancing driver safety and HMI design. *SAE Brasil 2025 Congress*.

Pan, S., Ge, Y., & Sun, X. (2025). A First Look at Privacy Risks of Android Task-executable Voice Assistant Applications. *arXiv preprint*.

Povey, D., Ghoshal, A., Boulianne, G., Burget, L., Glembek, O., Goel, N., Hannemann, M., Motlicek, P., Qian, Y., Schwarz, P., Silovsky, J., Stemmer, G., & Vesely, K. (2011). The Kaldi speech recognition toolkit. IEEE 2011 Workshop on Automatic Speech Recognition and Understanding. https://doi.org/10.21437/Interspeech.2011-154

Pong, H. K., Tai, Z. Q. C., & Choo, Y. L. (2025). Enhanced Malay Automatic Speech Recognition Through Fine-Tuning Pre-trained Whisper Models. *IFMBE Proceedings vol. 128*, pp. 558–566. https://doi.org/10.1007/978-3-031-97578-3_50

Shi, Q., Oztekin, A., Matthew, G., Bortle, J., Jenkins, H., Wong, S., Langlois, P., Zaki, A., Coleman, B., Luzuriaga, K., & Zai, A. H. (2025). GrantCheck an AI Solution for Guiding Grant Language to New Policy Requirements: Development Study. *JMIR Formative Research*, 9, e79038.

Shi, Y., Yu, W., Yao, W., & Liu, N. (2025). Towards Trustworthy GUI Agents: A Survey. *arXiv preprint*.

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing, 1*(2), 146-160. https://doi.org/10.1137/0201010

Torkamani, M. J., & Zarin, I. (2025). Adaptive edge-cloud inference for speech-to-action systems using ASR and large language models (ASTA). *arXiv preprint*.

Vu, M. D., Wang, H., Li, Z., & Chen, C. (2023). Voicify your UI: Towards Android app control with voice commands. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*.

Wang, F., Zhang, Z., Zhang, X., Wu, Z., Mo, T., Lu, Q., ... & Wang, S. (2024). A Comprehensive Survey of Small Language Models in the Era of Large Language Models:

Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness. *arXiv preprint*.

Wang, Y., Zhang, Z., Zhou, W., Zhang, W., Zhang, J., Zhu, Q., ... & He, J. (2026). GUIGuard: Toward a General Framework for Privacy-Preserving GUI Agents. *arXiv preprint*.

World Wide Web Consortium. (2026). Guidance on applying WCAG 2.2 to mobile applications. W3C. https://www.w3.org/TR/wcag2mobile-22/

Xie, N., Ni, Y., Liu, X., Cao, Y., & Chen, W. (2021). Implementation of a simulation control automation tool based on Android accessibility service. *Journal of Physics: Conference Series*.

Yang, Q., Li, H., Zhao, H., Yan, X., Ding, J., Xu, F., & Li, Y. (2025). FingerTip 20K: A Benchmark for Proactive and Personalized Mobile LLM Agents. *arXiv preprint*.

Ye, J., Zhang, X., Xu, H., et al. (2025). Mobile-Agent-v3: Fundamental Agents for GUI Automation. *arXiv preprint*.

Zhang, X., de Greef, L., Swearngin, A., White, S., Murray, K., Yu, L., ... & Bigham, J. P. (2021). Screen Recognition: Creating Accessibility Metadata for Mobile Applications from Pixels. *CHI Conference on Human Factors in Computing Systems*.

Appendix A

RAD Model Lifecycle

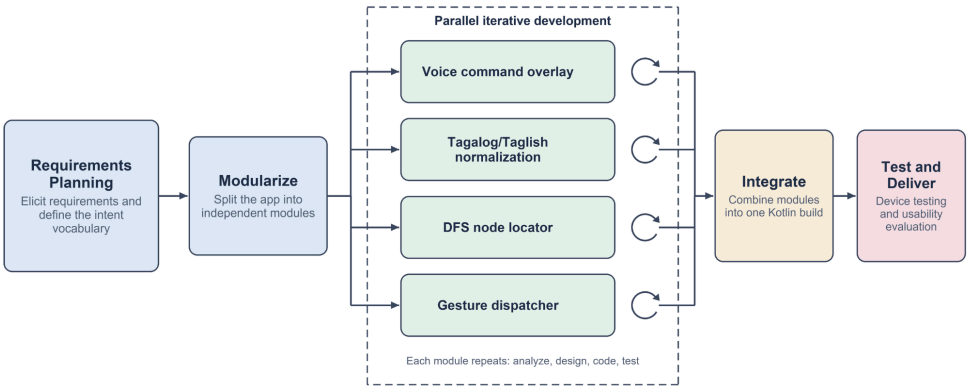


Figure 2 illustrates the Rapid Application Development (RAD) lifecycle applied to the HandsFree Android overlay construction. The model begins with requirement elicitation and modularization, after which parallel development teams independently build modular application components covering the voice command overlay, the Tagalog/Taglish normalization, the DFS node locator, and the gesture dispatcher, each repeating its own analyze, design, code, and test cycle, before integration into one Kotlin build and final device testing and usability evaluation. In terms of the four RAD phases described in the Methodology, Requirements Planning corresponds to the requirements planning phase, Modularize to the user design phase, parallel iterative development and Integrate to the construction phase, and Test and Deliver to the cutover phase. This iterative approach is selected because the dynamic constraints of the Android Accessibility Service API require continuous prototyping and testing rather than rigid upfront specification.

Appendix B

CRISP-DM Lifecycle

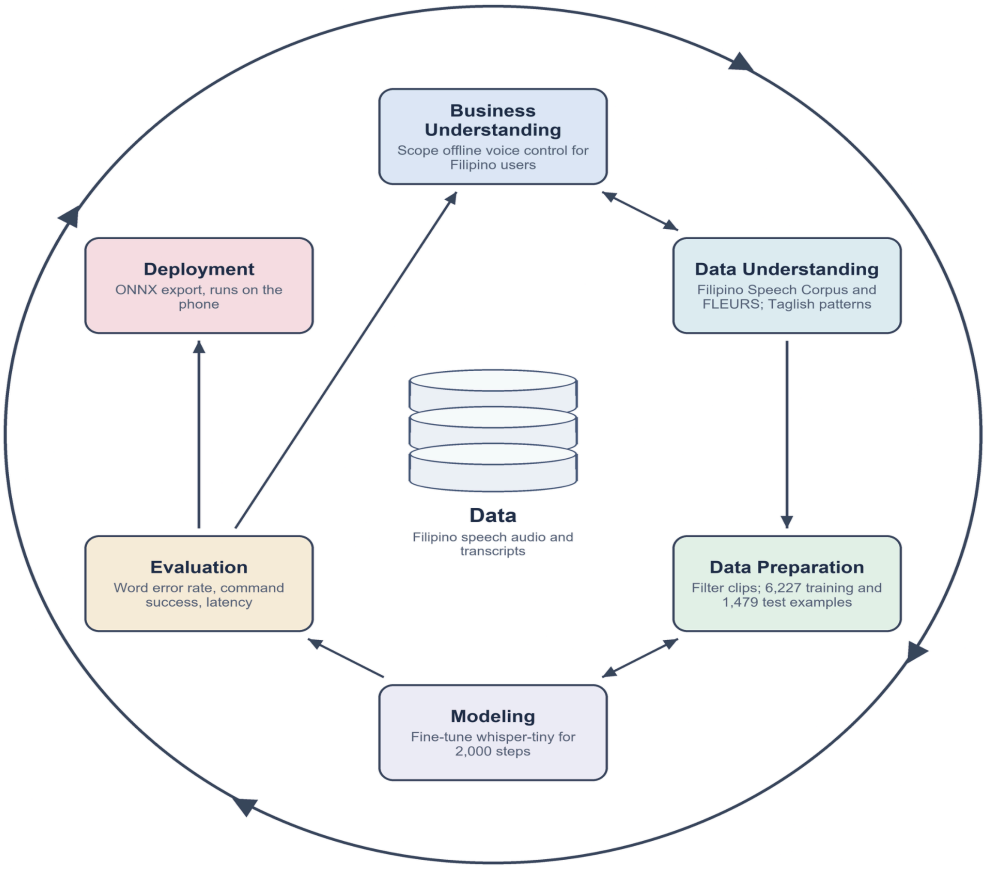


Figure 3 presents the CRISP-DM lifecycle governing the HandsFree Speech-to-Intent machine learning pipeline. The cyclical process begins with business and data understanding, proceeds through data preparation (filtering the Filipino Speech Corpus and FLEURS into 6,227 training and 1,479 test examples) and modeling (fine-tuning whisper-tiny for 2,000 steps), and culminates in evaluation (word error rate, command success, and latency) and

deployment (ONNX export so the model runs on the phone). The Levenshtein Distance matching that follows transcription belongs to the application pipeline, not to model training. The iterative nature of the framework allows the model to be refined based on empirical latency and accuracy measurements collected during constrained hardware testing.

Appendix C

Proposed Architectural Design of HandsFree

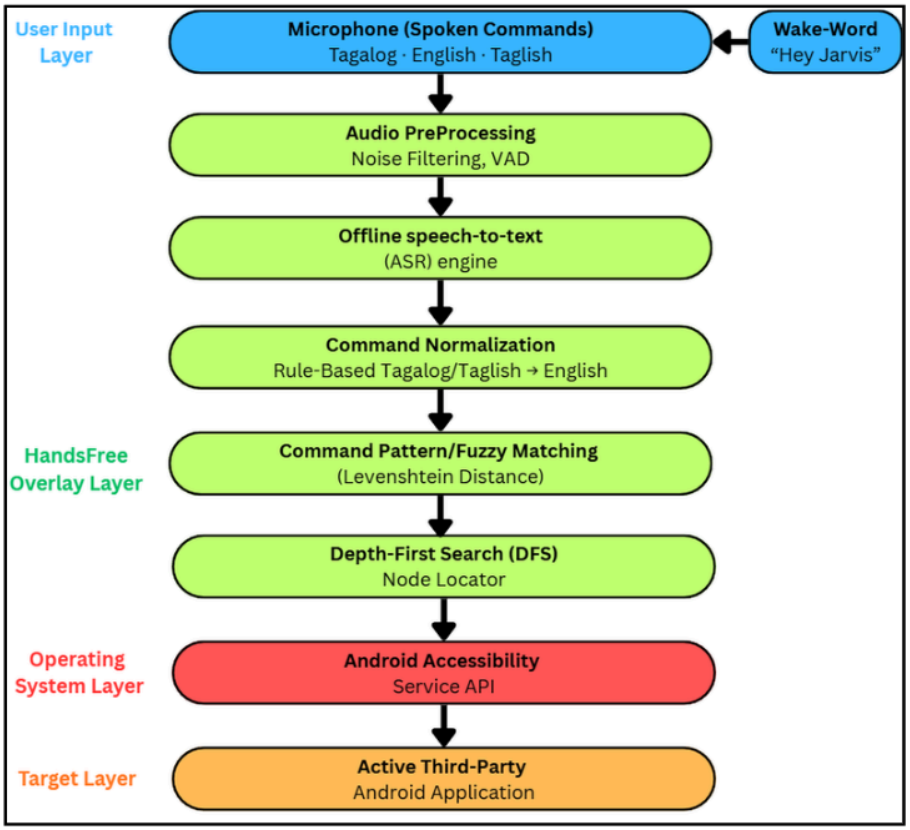


Figure 4 presents the proposed architectural design of the HandsFree system, organized into four distinct functional layers. The User Input Layer acts as the initial gateway, capturing raw audio via the microphone in Tagalog, English, or mixed Taglish registers. With the optional Call Sign enabled, the system continuously monitors for the "Hey Jarvis" wake phrase using a three-stage on-device cascade based on the openWakeWord architecture prior to full command acquisition. Within the HandsFree Overlay Layer, five sequential components process the signal: the Preprocessing Stage isolates speech through noise filtering and activity detection; the Offline Speech-to-Text (ASR) Engine performs

on-device transcription; the Rule-Based Tagalog/Taglish Command Normalization maps multilingual tokens to English counterparts; the Command Pattern/Fuzzy Matching stage utilizes Levenshtein Distance to resolve intents against a Predefined Command/Intent Vocabulary; and the Depth-First Search (DFS) Node Locator traverses the active view hierarchy to identify the target UI element. The Operating System Layer leverages the Android Accessibility Service API to extract structural interface metadata and execute programmatic gestures. Finally, the Target Layer represents the foreground third-party application receiving these simulated events, functioning universally without the need for developer-side integration or source code modification.

Appendix D

End-User Survey Instrument (Google Forms)

HandsFree Application — End-User Usability Evaluation

Thank you for participating in this evaluation. This survey uses the System Usability Scale (SUS) to assess your experience with the HandsFree application. Please answer each item based on your experience during the testing session. There are no right or wrong answers.

Section 1: Participant Background

1. Age: _____
2. Do you currently use any voice control or hands-free features on your smartphone?
□ Yes □ No
3. How often do you use voice control features on your device?
□ Never □ Rarely □ Sometimes □ Often □ Always

Section 2: System Usability Scale (SUS)

For each statement below, indicate your level of agreement on a scale from 1 (Strongly Disagree) to 5 (Strongly Agree).

Statement	1	2	3	4	5
1. I think that I would like to use this system frequently.	□	□	□	□	□
2. I thought the system was easy to use.	□	□	□	□	□
3. I found the various functions in this system were well integrated.	□	□	□	□	□
4. I would imagine that most people would learn to use this system very quickly.	□	□	□	□	□
5. I felt very confident using the system.	□	□	□	□	□
6. I needed to learn a lot of things before I could get going with this system.	□	□	□	□	□
7. I think that I would need the support of a technical person to use this system.	□	□	□	□	□
8. I thought there was too much inconsistency in this system.	□	□	□	□	□
9. I found the system unnecessarily complex.	□	□	□	□	□

10. I found the system very cumbersome to use.	□	□	□	□	□

Section 3: Open-Ended Feedback

1. What did you find most helpful about the HandsFree application?

_____

2. What challenges did you encounter while using the application?

_____

3. Would you recommend this application to others who could benefit from hands-free mobile control? □ Yes □ No
Why or why not? _____

Appendix E

Expert Evaluator Survey Instrument (Google Forms)

HandsFree Application — Expert Technical Evaluation

This instrument is intended for mobile application developers and Android accessibility professionals. Please evaluate the HandsFree application based on your technical expertise. Rate each item from 1 (Strongly Disagree) to 5 (Strongly Agree).

Section 1: Expert Background

1. Professional role: _____
2. Primary area of expertise in mobile development: _____
3. Years of experience in mobile application development with Android:
□ 1-3 years □ 3-5 years □ 5 years or more

Section 2: Technical Accuracy (Items 1–5)

Statement	1	2	3	4	5
1. The voice command recognition demonstrated acceptable accuracy for its intended use.	□	□	□	□	□
2. The gesture dispatch (tap/swipe/scroll) executed correctly and consistently.	□	□	□	□	□
3. The system responded within an acceptable time frame for real-time use.	□	□	□	□	□
4. The offline processing architecture is technically sound and reliable.	□	□	□	□	□
5. The Levenshtein-based fuzzy matching sufficiently accommodates natural speech variation.	□	□	□	□	□

Section 3: Mobile Usability and Design Standards (Items 6–10)

Statement	1	2	3	4	5
6. The application aligns with established general Android mobile usability standards.	□	□	□	□	□

7. The system is usable across a range of general Android users with varying technical familiarity.	□	□	□	□	□
8. The system is functional across different interaction styles and device types.	□	□	□	□	□
9. The privacy-first offline design is appropriate for accessibility use cases.	□	□	□	□	□
10. The system operates without requiring modifications to third-party applications.	□	□	□	□	□

Section 4: Practical Development Utility (Items 11–15)

Statement	**1**	**2**	**3**	**4**	**5**
11. The system is suitable for general daily mobile use to a broad user base.	□	□	□	□	□
12. The system demonstrates practical value as a voice-driven productivity tool.	□	□	□	□	□
13. The system reduces reliance on manual touch interaction for routine device tasks.	□	□	□	□	□
14. The system's modular design allows for practical expansion of supported voice commands without significant redevelopment.	□	□	□	□	□
15. Overall, the HandsFree system demonstrates viability as a practical voice-driven mobile application.	□	□	□	□	□

Section 5: Open-Ended Expert Feedback

1. From a technical perspective, what is the most significant strength of this system?

_____

2. What improvements would you recommend before public deployment?

_____

Appendix F

Informed Consent Form

Research Participant Informed Consent

Study Title: HandsFree: Development of a Deterministic Voice-Controlled Overlay for Android Navigation Using Accessibility Services

Introduction:
You are invited to participate in a research study aimed at evaluating a hands-free voice-controlled overlay application for Android devices.

Purpose of the Study:
The study aims to assess the usability and accuracy of the HandsFree application, which allows users to navigate Android devices without physical touch.

Procedures:
You will be asked to perform a series of standard tasks using the HandsFree application. This will take approximately 15-20 minutes.

Confidentiality & Data Privacy:
Your participation is voluntary. All data collected, including usability survey results, will be anonymized. No audio or personal identifiers will be retained. The application does not require network access and does not transmit data.

Right to Withdraw:
You may withdraw from this study at any time without penalty. Withdrawal can be completed by simply closing or uninstalling the application.

Statement of Consent:
By proceeding with the application evaluation, you acknowledge that you have read this information and agree to participate in the study under the conditions described.

Appendix G

Standardized 52-Command Test Set

This test set was utilized for the system's functional evaluation. It comprises 52 standardized voice commands spanning five core functional categories. This set includes a mix of English, Tagalog, and Taglish inputs to validate the system's Rule-Based Tagalog/Taglish Command Normalization component.

1. Application Launching

1. "Buksan ang YouTube"
2. "Open Facebook"
3. "I-open ang Settings"
4. "Buksan ang Messenger"
5. "Open Camera"
6. "Buksan ang Calendar"
7. "Open Calculator"
8. "Buksan ang Chrome"
9. "Open Spotify"
10. "Buksan ang Gallery"

2. Screen Navigation

1. "Bumalik"
2. "Go back"
3. "Pumunta sa Home Screen"
4. "Go to home"
5. "Mag-scroll pababa"
6. "Scroll down"
7. "Scroll pataas"

8. "Scroll up"
9. "I-scroll ang screen"
10. "Pindutin ang screen"
11. "I-tap ang button"
12. "Go to previous page"

3. Search & Information

1. "Hanapin ang adobo recipe"
2. "Search for news"
3. "Maghanap ng weather update"
4. "Search Google"
5. "Hanapin ang malapit na hospital"
6. "Search for restaurants"
7. "Hanapin ang movie schedule"
8. "Search lyrics"
9. "Maghanap ng flight status"
10. "Search for definition of science"

4. Call Management

1. "Sagutin ang tawag"
2. "Answer call"
3. "Tanggihan ang tawag"
4. "Decline call"
5. "Reject"
6. "Sagutin"
7. "Tanggihan"
8. "Answer phone"
9. "End call"

10. "Cancel call"

5. Scheduling

1. "Mag-set ng alarm"
2. "Set alarm for 7 AM"
3. "Set timer"
4. "Gumawa ng alarm"
5. "Set 5 minute timer"
6. "Alarm mamayang alas-otso"
7. "Set alarm 8 PM"
8. "Mag-set ng timer"
9. "Start timer"
10. "Stop timer"

Appendix H

Predefined Intent Vocabulary

The HandsFree system operates on a deterministic mapping of normalized voice input to specific system intents. While certain intents are fully implemented for execution, others specifically Messaging are supported at the recognition level to facilitate future modular expansion.

Intent Category	Intent Identifier	Description
Launch	LAUNCH_APP	Opens the target application identified in the command.
Navigation	NAV_BACK	Executes a "Back" gesture.
Navigation	NAV_HOME	Executes a "Home" gesture.
Navigation	SCROLL_UP	Executes an upward scroll gesture.
Navigation	SCROLL_DOWN	Executes a downward scroll gesture.
Search	WEB_SEARCH	Dispatches the query to the device search provider.
Calls	CALL_ACCEPT	Executes an "Answer" gesture for incoming calls.
Calls	CALL_REJECT	Executes a "Reject" gesture for incoming calls.
Scheduling	SET_ALARM	Sets a system alarm for a specified time.
Scheduling	SET_TIMER	Starts a system timer for a specified duration.
Messaging	MSG_INTENT	**Recognition Only:** Classifies the intent to send or reply to messages.

Appendix I

Experts' Curriculum Vitae

Expert 1

Appendix J

Editor's Note

Appendix K

Plagiarism Certificate

Appendix L

Researchers Curriculum Vitae

Researcher 1



ERIEL BEN L. BAGUIO

+63-966-483-8100

ebbags05@gmail.com

30 Turin St., Villagio Real
Angeles City

Skills

- Languages
  - Dart
  - Python
  - SQL
  - HTML/CSS/JavaScript
  - Java

Technical Skills

- Object-Oriented Programming
- Data Structures and Algorithms
- Data Analysis and Management

Summary

Eager to learn and motivated student with experience in data organization and analysis. Looking forward to any opportunities to apply organizational and technical skills in a growth-oriented and professional environment.

Experience

Procrastimate (Task Manager App | Flutter)
2025 3rd Year College

- Assisted in the UI/UX Design, and wireframe for the structure of the app. With the app being a focus on improving time management and organizing tasks for users to avoid procrastination through gamified features like points and streaks.

Frequency - (Music Platform Website WEBCS Project | HTML & CSS)
2025 2nd Year College

- Developed a web-based music platform aimed at helping users to discover genres of music. The main focus on the front-end structure and responsive design for a user-friendly experience.

Education

Bachelor of Science in Computer Science — **2023-Present**
Holy Angel University
- 1st and 2nd Year College Dean's Lister

Senior High School — **2021-2023**
Holy Angel University 2014 - 2018
- With Honors

Junior High School — **2017-2021**
Holy Angel University

Researcher 2



KARL SHANE BERNARTE

CONTACT

+63 975 025 0074

karlshanegt@gmail.com

1710, Jesus St. Pulungbulu, Angeles City

PROJECTS

- HandsFree
- Procrastimate
- Hercurles Gym Website
- Library Management

SKILLS

- Project Management
- Teamwork
- Effective Communication
- Frameworks & Libraries
- Object-Oriented Programming
- Data Structure and Algorithm

LANGUAGES

- Python
- Java
- C#
- SQL
- Dart
- HTML
- CSS
- Javascript
- Flutter

PROFILE

I'm passionate about continuous learning and growth, and I actively seek out opportunities to expand my skills and knowledge. I work well with others, stay open-minded, and believe every challenge is a chance to improve. I enjoy stepping into unfamiliar situations because they push me to think differently and develop new ways of solving problems. Whether working independently or as part of a team, I try to bring the same curiosity and dedication to every task I take on.

EDUCATION

Bachelor of Science in Computer Science — 2023- 2027 (IN PROGRESS)
HOLY ANGEL UNIVERSITY
- 2ND and 3rd Dean Lister

Senior High School (STEM) — 2021-2023
HOLY ANGEL UNIVERSITY

Junior High School — 2017-2021
HOLY ANGEL UNIVERSITY

WORK EXPERIENCE

Virtual Assistant
- Managed scheduling, email correspondence, and data entry tasks for clients, ensuring timely turnaround on daily requests
- Organized and maintained digital files/documents using Google Workspace and Notion

Webpage Management
- Responded to comments and messages from followers/customers, handling inquiries and basic customer service
- Monitored page insights/analytics (reach, engagement, follower growth) and adjusted content strategy based on performance
- Handled page moderation, including managing spam, inappropriate comments, or reviews
- Coordinated with business owner on content direction, campaigns, or announcements
- Handled page moderation, including managing spam, inappropriate comments, or reviews

Researcher 3



JUAN CARLO C. DE CASTRO

About Me

An upcoming bachelor with a Computer Science degree with a fascination of computers, eager to learn more how programs, graphics, data, and artificial intelligence interacts with computers and other IOT devices. Eager to collaborate and work with others in coding projects

09234023668

1carlodcastro@gmail.com

Village 2, Dela Paz Sur, City of San Fernando Pampanga

Language

- English
- Tagalog

Expertise

- Fast Learner
- Critical Thinking
- Problem Solver
- Python
- Java
- HTML & CSS
- Flutter & Dart
- Basics in Git & Github
- Computer Fundamentals
- Kotlin

Projects

HandsFree - Local Assistant for your phone
4th Year Capstone Project for CSSTUDY2
Created using a finetuned onnx model and possesses basic assistant features without the use of an internet. Created for English and Tagalog android users finding apps, scheduling, answering calls and much more

Procastimate - Productivity Management App
3rd Year Finals Project for 6IMSOFTENG & 6ADET
Created a productivity app for keeping tasks open implemented and managed Firestore and Firebase schematics to the app, allowing full authentication sign in, sign out, along with Firestore json management to store user values for main functions to work.

Catnip Cafe - A Reponsive Website
2nd Year Finals Project for 6WEBCS
Created a responsive website output to accomplish the final deliverable for Introduction to Web Development for Computer Science (6WEBCS), a website that simulates an online restaurant menu for a theoretical business (Catnip Cafe), complete with different features such as menu and photo gallery.

Education

Bachelor of Science in Computer Science **2023 - 2027**
Holy Angel University
- College Dean's Lister

Senior High School **2021-2023**
Holy Angel University
- With Honors

Skills Summary

Front-End Development 60%
Back-End Development 40%
Collaboration/Teamwork 75%

Researcher 4



JOHN HAROLD DOTON

MY CONTACT

Phone
+63-936-029-6134

Email
harolddoton@gmail.com

Address
SummerField Homes, Bl6 lot4, Bulaon SanRafael

EXPERTISE

- Programming
- Front-End
- BackEnd
- AI/ML
- Web Design

SOFT SKILLS

- **Self-Directed Learning**
- **Adaptability**
- **Time Management**
- **Communication**
- **Problem Solving**

ABOUT MYSELF

I am a Computer Science student at Holy Angel University focused on artificial intelligence, software engineering, and building things people actually use. I like turning ideas into AI tools and clean interfaces, and I occasionally like creating silly projects that I can turn into real working code.

Projects

Luna — Autonomous AI VTuber
Python, Groq LLaMA, edge-tts, Twitch IRC, TikTokLive API

- Developed a real-time conversational AI VTuber with live streaming integration on Twitch and TikTok
- Architected a two-tier RAG memory system to maintain short-term and long-term conversational context

SmartSort — Desktop File Organization Utility
Python, pywebview, HTML/CSS/JS

- Developed and packaged a standalone desktop application for automated file organization
- Designed a custom UI with real-time logging, dry-run previews, and undo/history functionality

Personal Portfolio Website
React, React Router

- Designed and deployed a personal developer portfolio hosted on Vercel

EXPERIENCE

July 2024 — April 2025

GDG on Campus · Holy Angel University — Internal Relations Officer - Handled the internal relations of the organization for GDG on Campus - Holy Angel University.

January 2025 — Present

Bachelor of Science in Computer Science — A Computer Science student with a passion for software development and technology.

Appendix M

Algorithm Flowcharts

Figure 8 shows how a normalised command is matched to an intent using the Levenshtein distance, and Figure 9 shows how the depth-first search finds the control the user named. Both are described in System Algorithms in the Research Procedure.

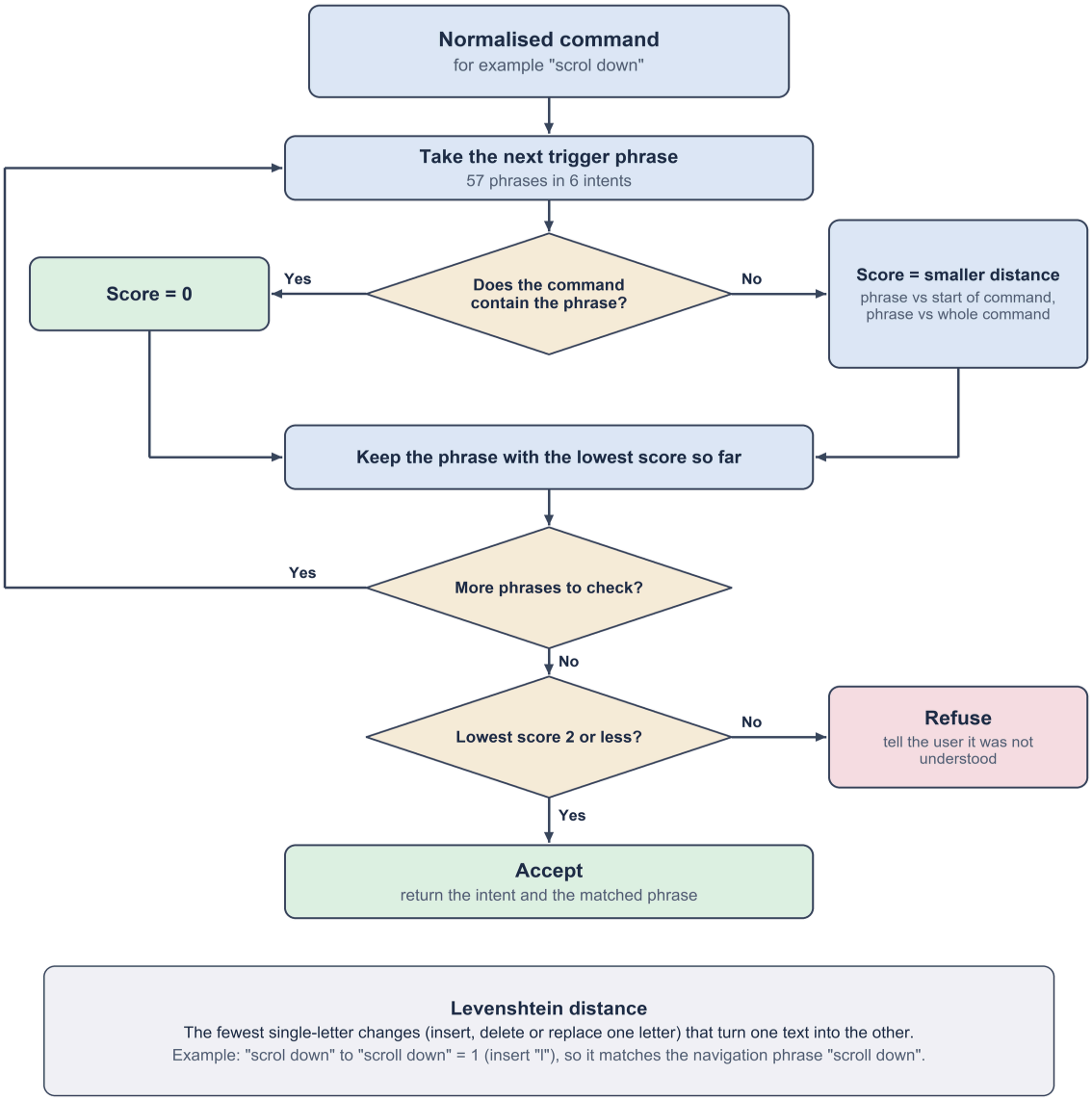


Figure 8 *How HandsFree matches a spoken command to an intent*

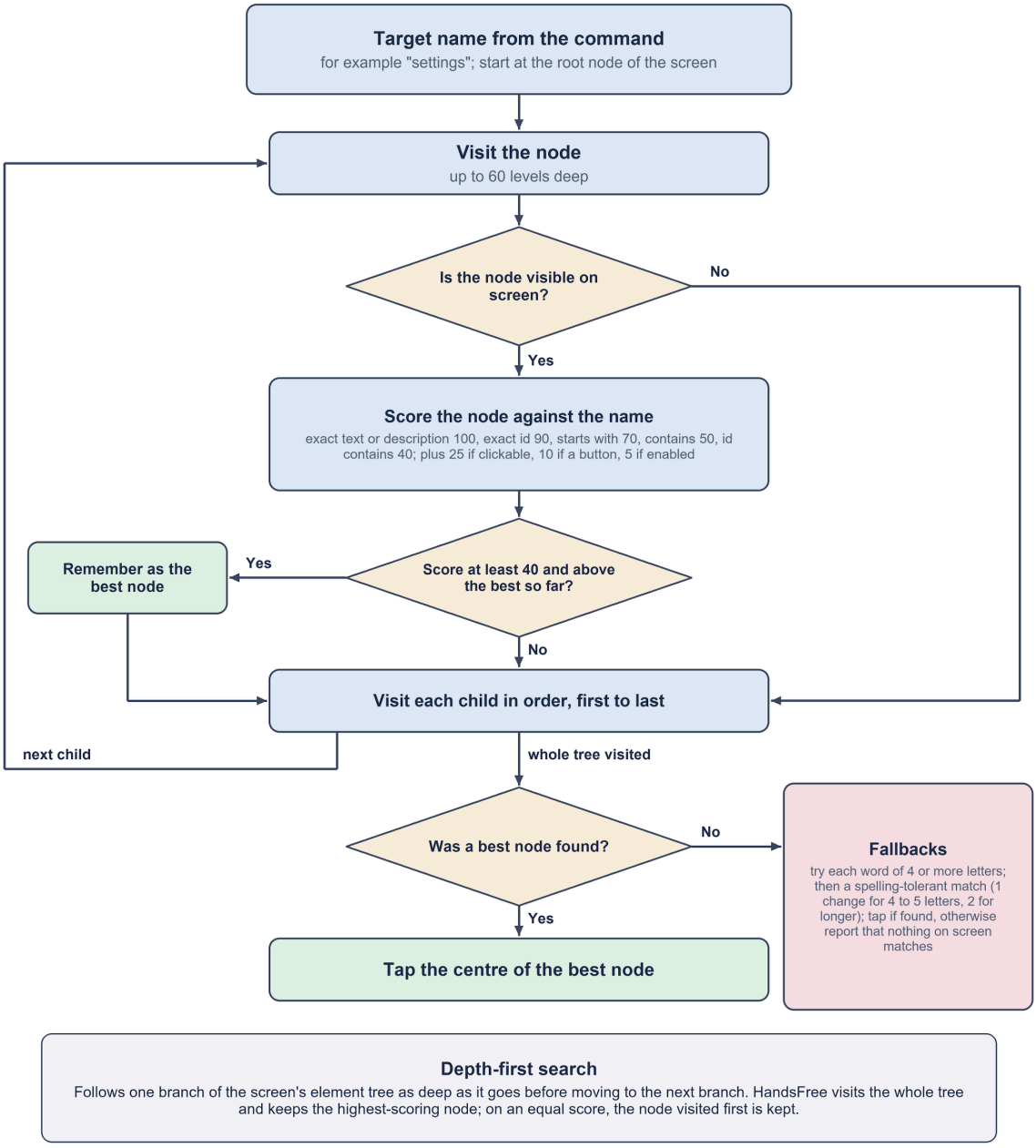


Figure 9 *How HandsFree finds the named control on screen (depth-first search)*

Appendix N

Source Code and Build Instructions

Repository. The source code of HandsFree is published at https://github.com/JCDC0/HandsFree. The repository holds the Android application source code in Kotlin, the bilingual normalisation lexicon and the intent vocabulary as JSON files, the wake-word models, the Docker configuration, and this manuscript in the docs folder.

Model files. The fine-tuned Whisper-tiny model, exported to ONNX as an encoder file, a decoder file and a token list, is published as the release named models-v1 in the same repository. The files are not stored with the source code because the decoder file (109 MB) is larger than the 100 MB file limit of GitHub. The original model weights are published at https://huggingface.co/KarlShane11/handsfree-whisper-tiny-tl.

Building with Docker. An Android application cannot run inside a Docker container, because it needs a phone, a microphone and the Android Accessibility Service. The Docker configuration therefore packages the build environment: Java Development Kit 17, Android SDK 35 and the Gradle build tool. The build downloads the model files from the models-v1 release and checks each file against its SHA-256 checksum. On a computer with Docker and Git installed, the following commands produce the installable application package (APK) in a folder named out:

```
git clone https://github.com/JCDC0/HandsFree.git
cd HandsFree
docker build --target apk --output out .
```

Docker container. A ready-built container image is published at https://github.com/users/JCDC0/packages/container/package/handsfree under the image name ghcr.io/jcdc0/handsfree. It holds the APK built by the configuration above. On a computer with Docker installed, the following command copies the APK, named HandsFree-debug.apk, into the current folder without building it:

```
docker run --rm -v ${PWD}:/out ghcr.io/jcdc0/handsfree
```

Installing. Copy out/app-debug.apk to a phone that runs Android 11 or later and install it. Then grant the Microphone and Display over other apps permissions, and turn on HandsFree under Settings, Accessibility. The application needs no internet connection after it is installed.